

Cloud Computing Architecture

Semester project report

Group 059

Diego Gobbetti - 25-740-069

Ennio Jose Fattiboni Ferrara - 25-720-277

Systems Group
Department of Computer Science
ETH Zurich
May 15, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.

Divergence from the template can lead to subtraction of points on graded parts of the project.

- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.
- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.
- **Use of AI Tools:** The use of AI tools must adhere to [ETH regulations](#). **You take responsibility for the content you submit.** You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 1

Using the instructions provided in the project description, run memcached alone (i.e., no interference), and with each iBench source of interference (cpu, l1d, l1i, l2, llc, membw). For Part 1, you must use the following `mcperf` command, which varies the target QPS from 5000 to 80000 in increments of 5000 (and has a warm-up time of 2 seconds with the addition of `-w 2`):

```
$ ./mcperf -s MEMCACHED_IP --loadonly
$ ./mcperf -s MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -w 2 -t 5 \
    --scan 5000:80000:5000
```

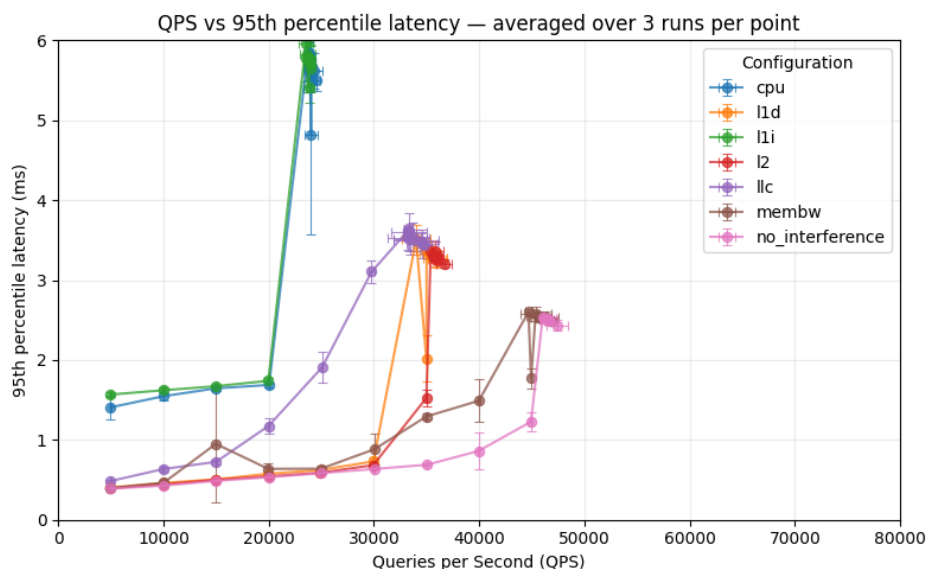
Repeat the run for each of the 7 configurations (without interference, and the 6 interference types) **at least 3 times** (3 should be sufficient in this case), and collect the performance measurements (i.e. the `client-measure` VM output).

Reminder: after you have collected all the measurements, make sure you **delete your cluster**. Otherwise, you will easily use up the cloud credits. See the project description for instructions how to make sure your cluster is deleted.

(a) Plot a line graph with the following stipulations:

- Queries per second (QPS) on the x-axis (the x-axis should range from 0 to 80K).
(**note:** the actual achieved QPS, not the target QPS)
- 95th percentile latency on the y-axis (the y-axis should range from 0 to 6 ms).
- Label your axes.
- 7 lines, one for each configuration. Add a legend.
- State how many runs you averaged across and include error bars at each point in both dimensions.

Answer:



- (b) How is the tail latency and saturation point (the “knee in the curve”) of memcached affected by each type of interference? What is your reasoning for these effects? Briefly describe your hypothesis.

Answer:

- **ibench-cpu:** The tail latency stays around 1.5ms until it reaches the saturation point at 25K QPS, where it reaches a tail latency of around 5.5ms

Explanation: CPU interference will affect the latency regardless of the number of QPS. But at some point the CPU gets completely overloaded and latency spikes

- **ibench-l1d:** The tail latency stays unaffected, until it reaches the saturation point at 35K QPS, where it reaches a tail latency of around 3.5ms

Explanation: Interference in the L1d cache (Level 1 Data Cache) does not increase latency until the cache is completely overloaded and stops being able to serve each process efficiently.

- **ibench-l1i:** The tail latency stays around 1.5ms until it reaches the saturation point at 25K QPS, where it reaches a tail latency of around 5.5ms. An effect similar to the previous CPU interference.

Explanation: The L1i cache (Level 1 Instruction Cache) affects latency because it slows the system regardless of the amount of QPS. But at some point the instruction cache gets completely overloaded and latency spikes

- **ibench-l2:** The tail latency stays unaffected, until it reaches the saturation point at 36K QPS, where it reaches a tail latency of around 3.5ms. An effect similar to the previous L1d interference.

Explanation: L2 cache acts generally as a fallback for L1d cache, so we can expect the effects to be similar.

- **ibench-llc:** The tail latency is affected at every step, with the tail latency growing gradually at every step from unaffected to around 3.5ms at the saturation point of 35K QPS

Explanation: The LLC (Last Level Cache) affects tail latency gradually as the probability of cache misses increases gradually with the number of QPS. Until it reaches a point where the computer can not handle more QPS.

- **ibench-membw:** The tail latency is unaffected until 35K QPS, where the tail latency starts increasing gradually until the saturation point of 45K QPS, the same saturation point and latency as the runs with no interference.

Explanation: The memory bandwidth only affects latency when the number of QPS starts causing LLC misses. Still, this kind of interference should not affect dramatically a service like memcached.

- (c) Processor affinity:

- Explain the use of the **taskset** command in the container commands for memcached and iBench in the provided scripts.

Answer: As stated in the [Linux manual page](#) for **taskset**

The taskset command is used to start a process with a given CPU affinity. CPU affinity is a scheduler property that "bounds" a process to a given set of CPUs on the system. The Linux scheduler will honor the given CPU affinity and the process will not run on any other CPUs.

CPU and core mean the same things.

- Why do we run some of the iBench benchmarks on the same core as memcached and others on a different core? Give an explanation for each iBench benchmark.

Answer: All of the iBench benchmarks run on the same core as memcached except for `ibench-llc` and `ibench-membw`. This is because we want to run CPU interference in the same core. And L1 and L2 cache interference also need to run in the same core as each core has their own L1 and L2 cache. Conversely, LLC and RAM memory is shared across all cores. To obtain a "purer" test we run the interference in another core to avoid also generating some CPU interference in the memcached core.

- (d) Assuming a service level objective (SLO) for memcached of up to 2 ms 95th percentile latency at 35K QPS, which iBench source of interference can safely be colocated with memcached without violating this SLO? Briefly explain your reasoning.

Answer: As shown by our experiments, we can colocate the memory bandwidth SoI as at and below 35K QPS, tail latency stays below 2ms. Tail latency only surpasses 2ms at 45K QPS, which is at the same load as no interference.

- (e) In the lectures you have seen queueing theory.

- Is the project experiment above (ignoring client measure) an open system or a closed system? Explain why.

Answer:

- What is the number of clients in the system? How did you find this number?

Answer:

- Sketch a diagram of the queueing system modeling the experiment above. Give a brief explanation of the sketch.

Answer:

- Provide an expression for the average response time of this system. Explain each term in this expression and match it to the parameters of the project experiment.

Answer:

- (f) Below we ask you to do a theoretical cost analysis of the above experiments you ran and compare with the actual cost of your experiments, for which you will **need to track the total runtime** of the experiments, from deploying the cluster to its deletion. To find the actual cost, you can look at the **detailed billing report** after it is available on your Google Cloud project.

For the theoretical one, it would be helpful to check the official pricing calculator ([link](#)) or given individual pricing list ([link](#)). You also need to see the `machineType` and `subnets` parameters of each VM configuration by examining `part1.yaml`.

Note you can see the actual configuration details (e.g. disk size and type) of each VM configuration by looking at each individual VM instance after you deployed your cluster in `Compute Engine > VM instances > <your vm name>`.

- What is the cost per hour (in USD) for running each VM, according to the VM pricing?

Answer:

- How much time (approximately) did your experiments take?

Answer:

- What is the expected (theoretical) total cost and what was the actual cost of running those experiments?

Answer:

- Does the expected cost match the actual cost? (i.e. equivalent ± 0.01 if rounded up to 2 decimals). If they do not match, explain how you could achieve a more accurate estimate of the cost.

Answer:

Part 2

1. Interference behavior

- (a) Fill in the following table with the normalized execution time of each batch job with each source of interference. The execution time should be normalized to the job's execution time with no interference. Round the normalized execution time to 2 decimal places. Color-code each cell in the table as follows: **green** if the normalized execution time is less than or equal to 1.3, **orange** if the normalized execution time is over 1.3 and less or equal to 2, and **red** if the normalized execution time is greater than 2. The coloring of the first three cells is given as an example of how to use the cell coloring command. **Do not change the structure of the table. Only input the values inside the cells and color the cells properly.**

Workload	none	cpu	l1d	l1i	l2	llc	memBW
barnes	1.00	1.57	1.81	2.20	1.76	2.56	1.80
blackscholes	1.00	1.84	1.35	1.63	1.33	1.44	1.32
canneal	1.00	1.11	1.07	0.81	0.81	1.15	0.73
freqmine	1.00	1.44	0.91	1.42	0.91	1.24	1.10
radix	1.00	0.93	1.05	1.07	1.05	1.44	0.96
streamcluster	1.00	1.32	1.29	1.46	1.30	1.81	1.28
vips	1.00	1.75	1.98	1.82	1.84	1.86	1.83

- (b) Explain what the interference profile table tells you about the resource requirements for each job. Give your reasoning behind these resource requirements based on the functionality of each job.

Answer:

- **barnes**: Overall relatively sensitive to all interference types, with LLC ($2.56\times$) and L1i ($2.20\times$) causing the most severe degradation.
Explanation: barnes workload traverses an octree with irregular, pointer-chasing memory accesses. These accesses pollute the entire cache hierarchy — every cache miss propagates upward — making the job highly sensitive to contention at all levels. LLC interference is most damaging because the tree's working set spans a large memory footprint that relies on the LLC for temporal reuse. The significant CPU sensitivity reflects its compute-heavy force-calculation kernel.
- **blackscholes**: Likewise Overall relatively sensitive to all interference types ($1.32\text{--}1.84\times$), with CPU ($1.84\times$) and L1i ($1.63\times$) cache causing the most notable degradations.
Explanation: blackscholes solves the Black-Scholes PDE for options pricing: a purely data-parallel, arithmetic-intensive computation. Its primary resource is CPU throughput, explaining the highest sensitivity to CPU interference. The uniform moderate sensitivity across all cache levels reflects regular array iteration over large option datasets — any cache eviction adds memory traffic, but the predictable access pattern limits the worst-case impact.
- **canneal**: Nearly insensitive to all interference types; no measurement exceeds $1.15\times$ slowdown, and some show values below 1.0 (likely measurement noise).

Explanation: Canneal performs simulated annealing on a netlist with uniformly random memory accesses. Its working set far exceeds any cache level, so the workload is DRAM-latency bound regardless of cache state. Adding cache stressors has negligible incremental effect because the caches are already ineffective for canneal’s access pattern, DRAM latency dominates under all conditions.

- **freqmine:** Sensitive to CPU ($1.44\times$) and L1i ($1.42\times$) interference, but resilient to L1d, L2, and membw ($0.91\times$) and LLC ($1.24\times$).

Explanation: **freqmine** mines frequent itemsets via the FP-growth algorithm, which involves intensive tree traversal with complex branching logic. This produces a large instruction footprint, explaining sensitivity to CPU and L1i contention. The compact in-cache data structure (FP-tree fits in smaller caches) explains resilience to L1d, L2, and memory bandwidth pressure

- **radix:** Resilient to all interference types except LLC ($1.44\times$); all other types cause $\leq 1.07\times$ slowdown.

Explanation: **radix** sort processes large integer arrays with regular, sequential access patterns. Accesses are stride-based and predictable, making prefetching effective and rendering L1/L2 and CPU interference irrelevant. However, the full dataset does not fit in the LLC, so LLC interference directly disrupts the sorting passes that rely on LLC for inter-pass data reuse, producing the only meaningful degradation.

- **streamcluster:** Relatively sensitive to LLC ($1.81\times$) and L1i ($1.46\times$), with mild sensitivity to CPU and L2 (≈ 1.30 – $1.32\times$), and resilient to L1d and membw (≈ 1.28 – $1.29\times$).

Explanation: **streamcluster** solves the online facility location problem through iterative distance computations over point sets. The algorithm’s computational kernels involve significant branching (facility assignment decisions), driving L1i sensitivity. The working set of active cluster centers and point data fits in the LLC but not smaller caches, making LLC eviction the primary performance bottleneck. Sequential scan over points keeps memory bandwidth and L1d pressure low.

- **vips:** Uniformly sensitive to all interference types (1.75 – $1.98\times$), with L1d causing the greatest slowdown ($1.98\times$) and CPU interference also significant ($1.75\times$).

Explanation: **vips** processes large images through a pipeline of pixel-manipulation operations (convolutions, colour transforms, etc.). Each operation streams large pixel buffers through L1 data cache repeatedly, making L1d the primary bottleneck and explaining the highest sensitivity there. The variety of filter kernels also creates instruction pressure (L1i: $1.82\times$). The large image data pollutes all cache levels, and the compute-heavy filters add CPU sensitivity

- (c) Which jobs (if any) seem like good candidates to colocate with memcached from Part 1, without violating the SLO of 2 ms P95 latency at 40K QPS? Explain why.

Answer: The best candidate is **canneal**. Its interference profile shows $\leq 1.15\times$ degradation under every stressor type, indicating it does not contend meaningfully for CPU cycles, instruction cache, data cache, LLC, or memory bandwidth in ways that would degrade co-located workloads. By symmetry, it is unlikely to evict memcached’s hot-key working set from the LLC or starve its request-processing threads of CPU cycles.

Also **Radix** is a partial candidate: it imposes negligible CPU, L1, L2, and membw pressure, which aligns well with memcached’s CPU- and LLC-sensitive nature. However, its LLC sensitivity ($1.44\times$) indicates it does compete for LLC capacity, which could elevate memcached’s cache-miss rate and increase tail latency under high QPS.

2. Parallel behavior

- (a) Plot a line graph with speedup as the y-axis (normalized time to the single thread config, $\text{Time}_1 / \text{Time}_n$) vs. number of threads on the x-axis (1, 2, 4 and 8 threads - see the project description for more details). Pay attention to the readability of your graph.

Answer:

Figure 1 shows the speedup of each PARSEC benchmark relative to its single-thread execution time, computed as T_1/T_n for $n \in \{1, 2, 4, 8\}$. In general, the benchmarks benefit from additional threads, although the scaling is not perfectly linear. **radix** shows the best parallel scaling, reaching a speedup of about $6.40\times$ at 8 threads, while **blackscholes** and **vips** flatten earlier, indicating diminishing returns at higher thread counts.

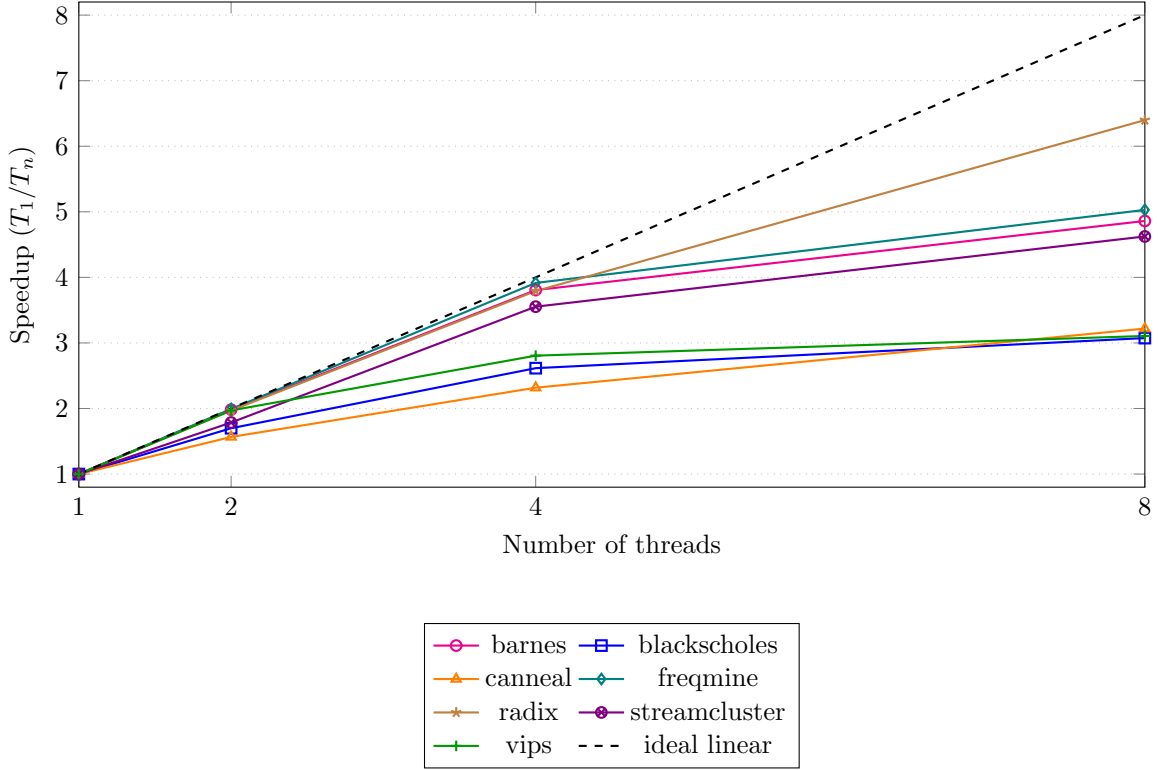


Figure 1: Speedup of PARSEC benchmarks as the number of threads increases from 1 to 8.

- (b) Briefly discuss the scalability of each job: e.g., linear/sub-linear/super-linear. Do the applications gain significant speedup with the increased number of threads? Explain what you consider to be “significant”.

Answer: We define a speedup as *significant* if it exceeds $1.5\times$ per doubling of threads (i.e., $>75\%$ parallel efficiency per step).

- **barnes**: Near-linear scaling from 1→4 threads ($3.81\times$ at 4T, 95% efficiency), then sub-linear at 8 threads ($4.86\times$, 61% efficiency). The gain from 1→4T is significant; from 4→8T the incremental improvement is modest due to synchronization overhead on the shared octree structure.
- **blackscholes**: Sub-linear throughout: $1.70\times$ at 2T (85%), $2.62\times$ at 4T (65%), $3.07\times$ at 8T (38%). The 1→4T gain is notable but far from linear; from 4→8T the gain is not significant. Memory bandwidth becomes a bottleneck as threads increase, limiting parallel throughput on the large options array.
- **canneal**: Sub-linear throughout: $1.57\times$ at 2T, $2.32\times$ at 4T, $3.22\times$ at 8T. No step achieves significant speedup. Canneal’s random memory accesses cause severe DRAM bandwidth contention as threads scale, since each additional thread independently generates random cache misses, quickly saturating memory bandwidth.
- **freqmine**: Near-linear at 2T ($1.99\times$) and 4T ($3.92\times$, 98% efficiency), then sub-linear at 8T ($5.03\times$, 63% efficiency). Speedup is significant and near-ideal up to 4 threads — **freqmine**’s FP-tree partitions well across threads with minimal cross-thread data sharing. The drop-off at 8T suggests synchronization or load-imbalance overhead at high thread counts.
- **radix**: Near-linear at 2T ($1.97\times$) and 4T ($3.79\times$), and approaching linear at 8T ($6.40\times$, 80% efficiency). This is the best-scaling benchmark. **radix** sort decomposes into independent per-digit passes with regular access patterns, minimizing inter-thread contention. The speedup is significant at every step.
- **streamcluster**: Sub-linear overall: $1.79\times$ at 2T, $3.55\times$ at 4T (89% efficiency), $4.62\times$ at 8T (58% efficiency). Scaling is significant from 1→4T; from 4→8T the gain (4.62 vs. 3.55 , $+1.07\times$) is modest. Synchronization on cluster centroid updates limits scaling at higher thread counts.
- **vips**: Near-linear at 2T ($1.97\times$), then sharply sub-linear at 4T ($2.81\times$, 70% efficiency) and barely improves at 8T ($3.11\times$, 39% efficiency). The only significant speedup step is 1→2T. **vips** processes images through a sequential pipeline of filter stages; once the pipeline is saturated, additional threads provide diminishing returns due to Amdahl’s-law limits on the serial pipeline coordination.

Part 3 [68 points]

1. [34 points] Describe and analyze your hand-crafted policy.
 - (a) [17 points] With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time ¹ across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency > 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes	42.33	0.58
blackscholes	43.00	0.00
canneal	119.00	2.65
freqmine	132.33	1.15
radix	37.33	0.58
streamcluster	125.67	3.06
vips	53.00	3.46
total time	247.67	4.04

Answer: We had 0 SLO violations throughout the 3 runs.

Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperrf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in **the color of vips** for core x from when vips is started to when it ends if vips ran on core x .

Plots:

¹Here, you should only consider the runtime, excluding time spans during which the container is paused.

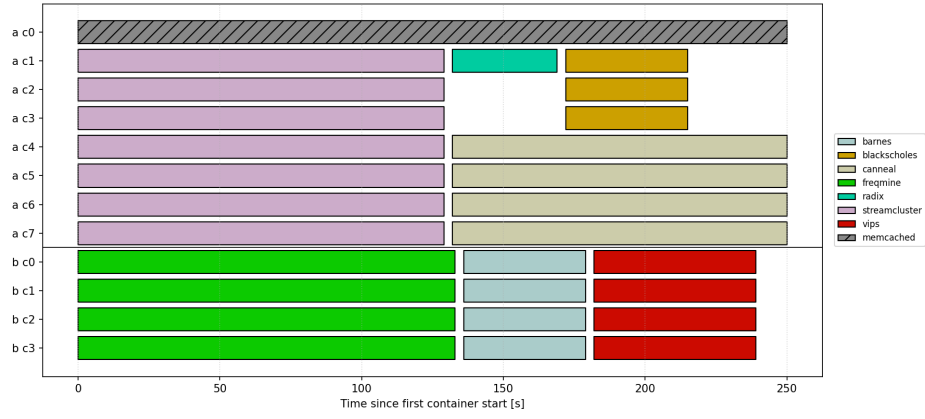


Figure 2: Run 1 schedule

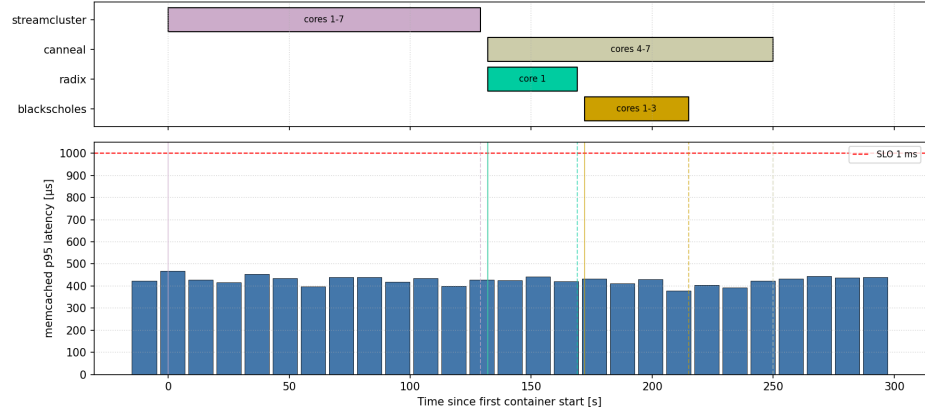


Figure 3: Run 1 memcache p95 latency

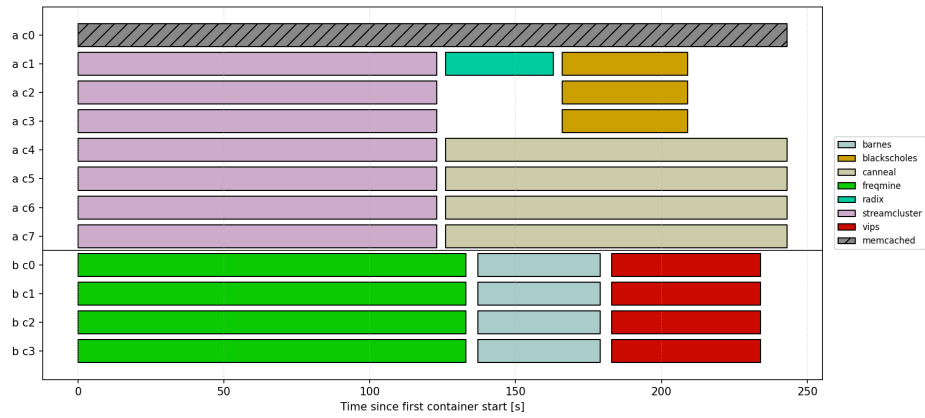


Figure 4: Run 2 schedule

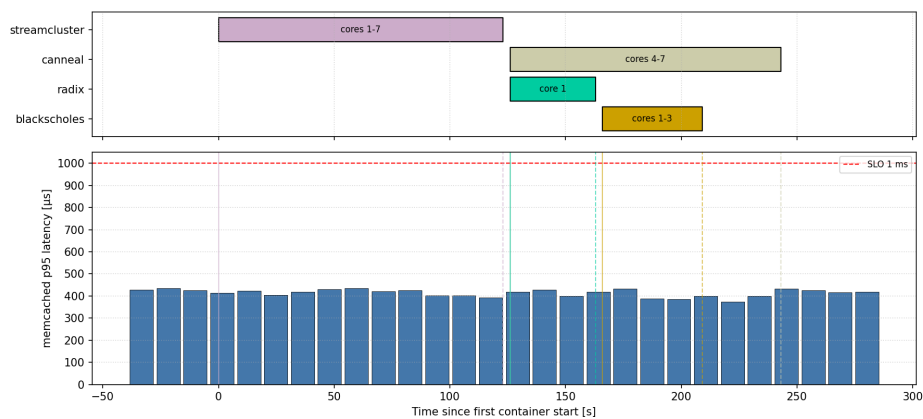


Figure 5: Run 2 memcache p95 latency

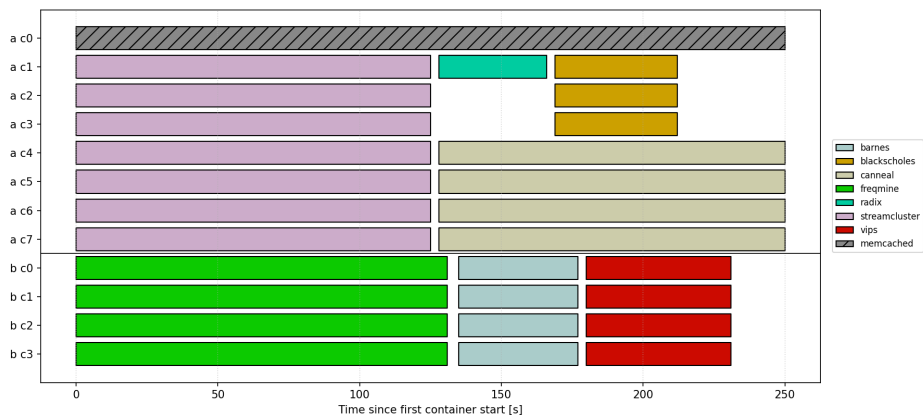


Figure 6: Run 3 schedule

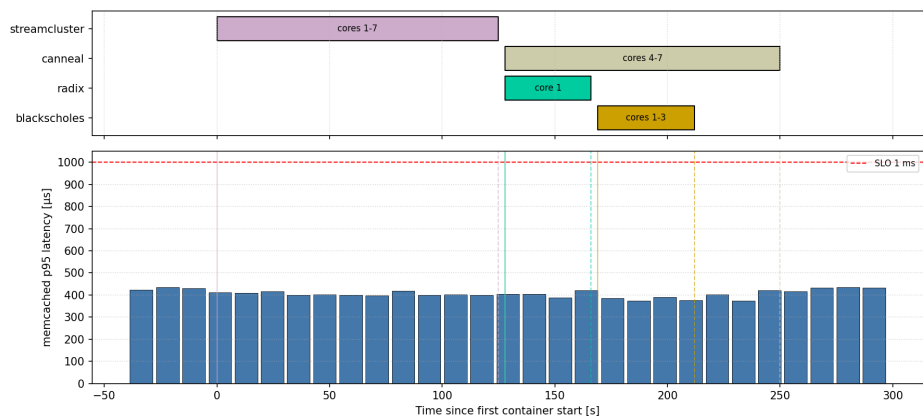


Figure 7: Run 3 memcache p95 latency

- (b) [17 points] Describe and justify the “optimal” scheduling policy you have designed.

We recommend starting reading from the last section, where we illustrate the principles on which we grounded the scheduling policy design.

- Which node does memcached run on? Why?

Answer: e2-standard-8. For multiple reasons:

- Accounting for its high sensitivity to CPU interference, it requires an isolated vCPU. Hence, eventually it is pinned to vCPU 0 and assigned one thread. The same allocation on **n2d-highcpu-4** would waste the potential leverage of brawny cores’ power, which could optimize primarily jobs with mostly serial workloads.
- Most importantly, when stressing the cache plane in further tests that saw memcached, pinned to a distinct core, and colocated with LLC-sensitive jobs, mainly vips and barnes, we observed consistently a spike of latency in the danger zone ($> 0.7ms$), instantly once either of the jobs or both was scheduled, in any core allocation setting, and on both nodes. As such, vips and barnes were physically isolated from memcached.
- **e2-standard-8**’s memory is 4x **n2d-highcpu-4**; which provides a solid guarantee on the low risk of memory interference.
- Moreover, further tests of jobs eventually scheduled on **e2-standard-8**, with one thread/vCPU fewer than the maximum available at their start time (e.g. streamcluster on 7 cores rather than 8), revealed that their deoptimization is minimal.

Finally, its placement is synergic and coherent with other job’s optimization levers used, mostly in terms of parallelization.

- Which node does each of the 7 batch jobs run on? Why?

Answer:

- **barnes: n2d-highcpu.** Barnes implements an N-body simulation via recursive octree traversal with irregular, pointer-chasing memory accesses that span multiple cache levels and cannot be prefetched. Part 2a directly quantified the consequence: LLC interference caused a $2.56\times$ slowdown, confirming that Barnes is intimately bound to LLC capacity for temporal reuse. Collocating Barnes on **e2-standard-8** with memcached would therefore directly threaten the latency SLO, as highlighted by the results of part 1: memcached 95th percentile latency gradually increases when LLC is interfered with, until it exceeds 1 ms at around 15KPS.
- **blackscholes: e2-standard-8.** Blackscholes is a purely data-parallel, arithmetic-intensive computation with regular array access patterns. Part 2a identified it as primarily CPU interference sensitive. Further experiments showed no significant runtime advantage at 4T on n2d (32.8s) over e2(43.0s) with equivalent thread counts. However, the **e2-standard-8**’s dynamic resource management introduces the risk of CPU allocation sharing between tenants. We accounted for this structural risk, particularly exacerbated since blackscholes is the most CPU-sensitive workload in the suite. Nevertheless, empirical measurements showed this effect is negligible in practice: several executions depicted a consistent makespan with close to zero deviation, as shown by the final reported results. We further care to state that this choice is consistent with our design

philosophy of accepting a bounded noisy-neighbor risk in exchange for reduced concurrent jobs interference risks.

- **canneal: e2-standard-8.** **canneal** performs simulated annealing with uniformly random memory accesses, making it rich of non-uniform DRAM accesses. Micro-benchmarking in Part 2a confirmed this: $makespan \leq 1.15x$ under every stressor type, exhibiting minimal interference sensitivity, as DRAM latency dominates. That’s the main reason for **canneal** being the ideal memcached co-tenant: it is very unlikely to evict memcached’s hot keys in LLC, and the memory footprint is relatively low. On n2d at 4T, canneal takes 91.0s, a substantial speedup compared to 119.00 on e2, however we keep faith to the non-negotiable SLO principle adopted in our design, and allocated in on **e2-standard-8**.
- **freqmine: n2d-highcpu.** Freqmine mines frequent itemsets via the FP-growth algorithm: a compute-intensive tree traversal with near-linear parallel scaling ($3.92\times$ at 4T in part 2b). Near-linear scaling compounds directly with the EPYC Milan processor architecture, cross-core IPC allows higher single-thread throughput, since a single workload unit can be spread across all threads. Further experiments on n2d confirmed that freqmine at 4T completes in 120.90s, considerably lower than 155s on e2, with the same threads count.
- **radix: e2-standard-8.** Radix was assigned to **e2-standard-8** by a hard constraint surfaced during experimentation: on **n2d-highcpu** consistently failed with OOMKilled fault, regardless of collocation, allocated cores and pinning, and kubernetes job configuration requests/limits on memory. In the research of the root cause, by reading SPLASH-2 original paper, we have found a few aspects that could contribute to this failure mode, and eventually asserted that the only reasonable technical explanation is that Radix’s algorithm, with the native dataset, involves a data manipulation procedure that may burst DRAM consumption, specifically a data amplification during a intensive permutation, and the risk is exacerbated if DRAM is contended. We are aware that this argumentation is quite abstract, but given the time constraints, we could not find a more concrete cause. With 32 GB of DRAM, 4 per vCPU, **e2-standard-8** accommodates radix without risks, although a residual negligible risk of noisy neighbour remains.
- **streamcluster: e2-standard-8.** Streamcluster is the longest-running job, considering an equal thread count comparison. As such, the design leverages as much as possible the computational capabilities of **e2-standard-8**, over which the total mean makespan eventually is 125.67s. Similar results were observed in experiments, which denoted that executing it on **n2d-highcpu-4** at full capacity would be suboptimal: mean makespan 163.6s.
- **vips: n2d-highcpu.** Vip’s engagement was very similar to Barnes’s. Although the workloads types are different, resource sensitivity from a memcached perspective is pretty much the same, as shown by Part 2a: broad sensitivity across all resource types, with L1d the most severe. Hence, placing it on **e2-standard-8** with memcached would pose a severe risk to the memcached latency SLO. Moreover, further experiments on n2d reported a vips’s makespan of 50.6s with 4 threads, which, compared to the mean of 84.9s over **e2-standard-8**, with equal threads, depicts a more than marginal speedup.

- Which jobs run concurrently / are colocated? Why?

Answer: First of all, initial experiments with concurrent jobs revealed that interference was more severe than the part 2a micro-interference benchmark predicted. We observed several behavioural inconsistencies among jobs that we could not map to a root cause within the time and knowledge constraints of the project. At this point in the design process, we re-grounded our approach on the principle of consistent jobs’ makespan. We re-based the design on a fully sequential schedule and monotonically added concurrent allocations, covering broadly the breadth of compatible co-location settings. Secondly, we asserted that running concurrently jobs with Guaranteed QoS covering all the cores yielded, inconsistently, an unreliable effective start time of at least one job, we expand on this problem in the final section. Throughout concurrency experiments, we asserted that most theoretically compatible job pairs degraded under concurrent execution rather than sequential. We re-ran the part 2a tests multiple times to verify the profiling. Although seemingly imprecise, the theoretical compatibilities pictured by part 2a profiling were approximately confirmed by empirical concurrent-job tests, once interpreted under the symmetric principle that high sensitivity to a resource implies heavy use of that resource. The following are the critical insights we used as building blocks in the design:

- Any job run concurrently with **barnes** or **vips** degrades significantly in any core allocation setting. By the mentioned symmetric principle, **barnes** (LLC $2.56\times$, L1i $2.20\times$) and **vips** (uniform $1.75\text{--}1.98\times$ across all resources) are broadly aggressive across the cache hierarchy and memory access bus. Empirical tests with **freqmine**, with various cores partitioning settings, confirmed the thesis: on average a $\sim 45\%$ slowdown compared to an isolated execution with the same cores/threads count, again since the LLC and DRAM access bus are physically shared across cores.
- **streamcluster** pairs poorly with any minimally LLC-sensitive job. Its $1.81\times$ LLC sensitivity reflects an active working set that lives in LLC, which translates into heavy LLC eviction pressure on co-tenants. We have attempted to pair it with jobs compatible with **e2-standard-8** allocation (low memcache interference), specifically with **radix** and **blackscholes**: across multiple experiments, we have revealed inconsistent mutual slowdowns in the range $\sim 25\text{--}40\%$, always compared to an isolated allocation setting.
- **blackscholes** and **freqmine** are denoted for their CPU-sensitivity ($1.84\times$ and $1.44\times$ respectively), implying high IPC and execution-unit demand. Even when pinned to disjointed cores, on n2d, their concurrent execution would result in conflicts over SMT-shared physical resources (l1, l2, TLB). Their collocation in this node was a research direction that we have paved on in the final experiments, since without **blackscholes**, **canneal** on **e2-standard-8** could harness the full computation capacity, however brief empirical tests revealed an average $\sim 35\%$ slowdown on both jobs when colocated, which eventually outweighed the **canneal** makespan reduction.
- **canneal** is the universal safe co-tenant. With $\leq 1.15\times$ sensitivity to every resource, **canneal** does not pose a threat of interference over any resource, besides collaterally for DRAM access, as mentioned above and showcased in the experiments at the end of this section. That’s the main reason for **canneal** being the ideal memcache co-tenant. Even though to be precise, further research

revealed that actually uniformly-random DRAM access, implies also LLC residency, which could conflict with memcached sensitivity. However, throughout several tests, a clash basically never surfaced; we have concluded that this is still due to the fact that DRAM access latency dominates, such that LLC accesses are less frequent and the time-averaged LLC eviction is bounded and can be supported by memcache.

- **radix** is the second-safest co-tenant: only LLC-sensitive ($1.44\times$), inert across all other resources ($\leq 1.07\times$), and short (12s mean makespan with 4T on e2). It pairs cleanly with **canneal** because it does not stress LLC, and radix’s short runtime bounds the LLC pressure window to a small fraction of the phase.

With that being said, eventually, only **canneal-radix** and **canneal-blackscholes** run concurrently. A choice of remarkable importance, in which was invested substantial research time, is on the **canneal-blackscholes** collocation setting. We bind **canneal** to cores 4–7 and **blackscholes** to cores 1–3 via taskset, making vCPU contention impossible by construction. Given that **canneal** is the phase bottleneck (4T on e2: 119s) and **blackscholes** completes well within that window (3T: 42s), we tested the intuitive optimisation of reallocating one core from **blackscholes** to **canneal**, in order to shorten **canneal**’s tail. The result was the opposite: **canneal** at 5T degraded to 140s and **blackscholes** at 2T degraded to 55s, worsening both dimensions despite the clean core partitioning. The root cause lies in the **canneal**’s memory access pattern: uniformly random accesses maximize DRAM bank conflicts, where each access targets an unpredictable bank, invalidates the previous row buffer, and forces a full precharge-activate-read cycle that the memory controller cannot pipeline across. Adding a fifth concurrent thread amplifies the rate of bank-conflicting requests pending, raising memory controller queue depth and effective per-access latency beyond what the extra parallelism recovers; as such the fifth thread degrades aggregate throughput rather than adding to it. Due to the same conflict also 3T/4T, 3T/3T and 4T/4T(with one overlapping core) tested splits revealed to be suboptimal. The 4T/3T split is therefore the empirical optimum for this phase, and we couldn’t find any alternative choice to optimize the **canneal** tail.

- In which order did you run the 7 batch jobs? Why?

Order:

e2-standard-8: **streamcluster** - **radix** && **canneal** - **blackscholes** (after **radix**)

n2d-highcpu: **freqmine** - **barnes** - **vips**

Why: the ordering choices are mostly relevant for **e2-standard-8**, where memcached run, and double-folded:

- Longest jobs first: following a canonical time-optimization scheduling design, the longest jobs in each node are run first. This is beneficial primarily on **e2-standard-8**, where, by running **streamcluster** first, the scheduling idle time before concurrent jobs is minimized. Otherwise, **streamcluster**, allocated to 7 cores, would have to wait for previous jobs to complete to execute at full capacity, or start sub-powered and then burst - not feasible if taskset-based CPU pinning is used.
- Prevent LLC pressure in the most critical phase: the alternative ordering(current one switched) suggested in the previous point comes with the drawback of multi-job traffic on LLC during the initial phase, which empirical tests have proved to

be severely critical for memcached: its latency increases more steeply if llc-bound jobs are executed during the initial phase of its lifecycle, compared to following phases, evidencing a data access pattern that gradually maps to deeper levels of caches, such that LLC is highly sensitive initially. Hence, coallocating only streamcluster with memcached in the first phase implies that only a single job's LLC working set competes with memcached's hot-key data during the longest and highest-LLC-pressure window of its lifecycle. Although part2a outlined streamcluster as rather sensitive to the LLC ($1.81\times$), with memcached taskset's pinned to core 0 configured with Guaranteed QoS that specifies 1 vCPU and 1 Gb of memory, potential initial LLC conflicts do not impact its latency significantly, as verified with multiple experiments that highlight consistently a very mild increase in latency once streamcluster is allocated, likewise, reported results show the same pattern.

On a side note: throughout multiple experiments, we have tested two alternative orderings of the eventually assigned jobs on `n2d-highcpu-4`, results depict negligible skews on total node makespans, which is actually due to underlying physical machine and virtualization non-deterministic factors, rather than the ordering itself.

- How many threads have you used for each of the 7 batch jobs? Why?

Answer: Thread numbers are equal to the job configuration limits vCPUs across all jobs

 - `barnes`: 4/4. On `n2d`, `barnes` scales from 170.0s at 1T to 43.6s at 4T, showing an almost linear speedup optimization, capturing the steepest region of the scaling curve. The 4 $\rightarrow$ 8T marginal gain is only $1.28\times$, and anyway, with the chosen placement, it is inaccessible given the 4-core node constraint.
 - `blackscholes`: 3/7. Part 2b shows `blackscholes` scaling sub-linearly on `e2` ($2.62\times$ at 4T, $3.07\times$ at 8T), reflecting memory bandwidth exponential decay. Nevertheless, its absolute makespan is relatively high, and the gain obtained at 3T ($2.33\times$) can't be neglected; as such, we assign 3T. It runs concurrently with `canneal`, with an unbalanced cores allocation toward `canneal`, which is more compute-intensive and eventually represents the concurrent bottleneck that determines when the second scheduling phase ends on `e2-standard-8`. Throughout empirical tests, we asserted that moving the weight of computational capabilities significantly to either of the two would severely diminish the other, and the total makespan; the final configuration is the sweet spot, with more details on this in the concurrency paragraph.
 - `canneal`: 4/7. `canneal` has the worst scaling across all jobs, as shown from part2b results: $1.57\times$ at 2T, $2.32\times$ at 4T, which outline the bottleneck being in the memory access saturation. This is rooted in the uniformly random DRAM access pattern that denotes its workload: no timing or spatial locality, resulting in a consistent TLB miss, and most importantly high spreadth of data across DRAM banks, resulting consistently in DRAM bank row precharge-activation triggering, such that the memory controller can't service simultaneously multiple conflicting requests. With that being said, multiple experiments revealed that allocating to `canneal` 4 threads/vCPUs was the optimal choice. Similarly to `blackscholes`, this is expanded in the concurrency paragraph.
 - `freqmine`: 4/4. `freqmine` scales almost linearly up to 4T ($3.92\times$, 98% efficiency), representing the highest parallel speedup optimization in the suite.

- **radix**: 1/7. Radix has the best parallel scaling up to 8T in the suite, but its very contained absolute runtime makes the optimization gain very minimal. The concurrent **canneal** optimization speedup outweighs radix’s one.
 - **streamcluster**: 7/7. Minimising as much as possible streamcluster’s makespan is a primary scheduling goal. Part 2b shows that the speedup is still steep at the 4→8T step (8T - 4.62×). Experiments have shown that at 7T the speedup is still quite meaningful (4.3x), such allocation grants the highest possible computational power and therefore reduces significantly in relative terms the longest job makespan.
 - **vips**: 4/4. vips scaling behaviour is quite significantly sub-linear: 1.97× at 2T, then flattens rapidly (2.81× at 4T, 3.11× at 8T); nevertheless, its total makespan is relatively high, capturing the 2.81× optimization at 4T is quite substantial.
- Which files did you modify or add and in what way? Which Kubernetes features did you use?

Answer: The benchmark was automated with a Python script, which encodes our scheduling policy and uses directly Kubernetes API leveraging the [official client library for Kubernetes](#) for starting and monitoring the jobs. The core algorithm involves traversing a DAG that encodes dependencies of each job, which can’t be scheduled unless its dependencies are completed. For example, in our final schedule, **streamcluster** and **freqmine** do not have any dependencies, while **vips** depends on **barnes** (and **freqmine**). Remarkable effort was put into implementing a job lifecycle management system that reduces as much as possible the windows between status check and scheduling, and between scheduling and effective start. Specifically, by means of pulling images earlier in nodes and decoupling job status check and scheduling components with an asynchronous program structure.

Particularly, the job monitoring component leverages the kubernetes Job abstraction and its V1JobStatus conditions (Complete and Failed), accessed through the BatchV1Api of the official Python client library. We adopt an event-driven paradigm via the `watch.Watch().stream()` primitive over `list_namespaced_job`, anchored on a tracked `resource_version` that guarantees continuity across reconnections and prevents missed events. A polling-based interaction would introduce an overhead between a job’s effective completion and the scheduler’s reaction, equal to the poll time in the worst-case. Streaming pushes state transitions to the scheduler as soon as the API server observes them, minimizing the dispatch window of DAG-successor jobs.

When it comes to kubernetes job configurations, during the initial experiments, we consistently observed a scheduling issue when running multiple jobs concurrently under the guaranteed QoS class: whenever the sum of the requested vCPUs matched the total number of vCPUs available on the node, at least one job consistently got stuck in the pending state waiting for resources. To investigate the issue, we compared the node capacity reported by Kubernetes with the resources that were actually allocatable for workloads. Specifically, this was verified empirically at runtime by inspecting node information and monitoring resource consumption during execution through commands such as `kubectl describe node`, `kubectl top nodes`, and `kubectl top pods`. This measurement revealed that the scheduler does not reason on the raw hardware capacity, but on the node allocatable resources, which

are always lower than the spec CPUs. We eventually concluded that the most plausible explanation was the reservation of CPU resources internally by the system. Specifically, a fraction of the available cores is consumed by operating system processes, kubelet components and background services deployed as daemonsets. Under concurrent settings, these conflicts lead the scheduler to wait the scheduling of some jobs because the effective allocatable CPU budget has already been exhausted. To avoid this condition, we adopted a slightly conservative resource configuration. For each job, the CPU limit was set equal to the exact number of threads assigned to the application, while the CPU request was configured to one core less. This approach preserves the possibility for the job to exploit the full amount of CPU resources when available, while at the same time preventing the scheduler from saturating the node allocatable capacity.

A remarkable role is played by taskset. We are aware that amending it could open to a more flexible scheduling, particularly in the final scheduling `canneal`, collocated with `radix` and `blackscholes`, without being pinned to specific cores, could burst to cover all available cores in multiple time frames. However, from the very first experiments we revealed that interferences were severely higher without CPU pinning and makespans were consistently decayed compared to a configuration with `ot`, and most importantly, makespans were not consistent, which clashes with one of our core design principles. We have also experimented with Kubernetes static CPU management as an alternative to taskset; however, given the different planes their mechanism are rooted in, CPU pinning with taskset always outworked the counterpart with static CPU management, therefore this research direction was dropped early on due to the time constraints. The benefits of taskset resolve to guaranteeing CPU, NUMA, and cache locality affinity, preventing costly thread migrations that induce cache trashing and LLC-level false sharing among co-located jobs. This minimizes shared-cache contention and interference, ultimately providing more stable and predictable makespans across executions.

Finally, only for `streamcluster` we set requests and limits also on memory, respectively to 1 and 30 GB, to prevent potential interferences with `memcached`, which occasionally surfaced during experiments.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer: Our design choices are grounded in critical insights from initial empirical tests that explored the space of potential solutions in various diverse directions, beyond insights from previous parts.

Our design is based on three principles:

- Non-negotiable `memcached` latency SLO - first priority: one p95 latency measurement above the threshold would invalidate the execution. Accounting for `memcached` being pinned to a specific core via taskset, we have attempted to design a schedule that would primarily reduce the residual risk of memory and caches' interferences, given its high sensitivity surfaced in part 1 tests results, without trading off significantly the leverage of concurrency and parallelization.

Following this philosophy, memcached runs **e2-standard-8**, which has more DRAM space and lower access time, compared to **n2d-highcpu**, with equal l1d and l1i size. However, we care to state that we were aware of the counter-productive aspects of this configuration, remarkably the virtual machine’s CPU designs: **n2d-highcpu** guarantees sharing of one physical core among two virtual ones via SMT, whereas the more cost-optimised **e2-standard-8**, besides SMT with an undefined amount of vCPUs per CPU, involves also dynamic resource management to share physical cores between multiple tenants, resulting in lower reliability on cache latencies. Nevertheless, throughout the all experimental phase, we have detected negligible interference from tenants sharing the same resources, or that we could attribute to it.

- Consistent jobs’ makespan: first experiments attempted to minimize the total makespan by running at least 2 jobs concurrently at each machine. We got worse performance than what we expected due to a more severe interference than predicted, and moreover, inherently non-deterministic, as such quite difficult to analyse soundly and produce reliable data to back decisions on. The main consequent impact was high inconsistencies of jobs’ makespans. Switching to a fully sequential baseline allowed us to achieve more consistent jobs’ makespans, over which we started building on.
- Guaranteeing requested resources: throughout initial tests we have asserted that concurrent jobs with Guaranteed QoS with a total allocation that covers all the available vCPUs was diminishing the overall makespan: at least one job was not scheduled until another completed. We realised that this scheduling failure was rooted in the amount of configured vCPUs not being available at the programmed scheduling time, since we never accounted for one vCPU being potentially used by other processes running on the virtual machine. The kubernetes scheduler left the job in pending state, waiting for resources to be freed. Driven by this insight, we have made the choice of adopting Burstable QoS for all the jobs, downgrading the requested cores to at least limit-1.

We are aware of the inherent drawback of the adopted design philosophy: concurrency leverage is minimal, potentially leaving some space for total makespan reduction.

As mentioned throughout previous answers, although we were limited by rather strict time constraints, we have executed some initial probe experiments, designed accounting for previous parts’ insights and the provided machine resources specifications. One class of the very first experiments involved running each job again isolated on **n2d-highcpu** in all possible vcpus allocation settings, on an average of three times and finally annotate the mean makespan in each setting, which we mentioned multiple time above e.g [here](#). The specification of **n2d-highcpu** details it as using AMD EPYC chips with brawny cores, hence tailored for compute-intensive workloads and a higher LLC size compared to the cost-optimised **e2-standard-8**. The main goal of these experiments was to measure the optimization speedup gained from running the job in the high-performance CPU, as all of our previous experiments have been in e2-standard machines. Another consequent class of experiments involved running multiple pairs of theoretically compatible jobs, looking for their potential gain over a sequential schedule, and noting down the relative slowdowns, which are mentioned above, e.g. [here](#) Moving on, we have also studied the workload

types and asserted that **vips**, **freqmine**, and partially **streamcluster** jobs are rich in serial bottlenecks, communication between threads, and stragglers. Thus, more suited for **n2d-highcpu**. This is supported by the synchronization analysis from the PARSEC characterization study: **vips** exhibits 33,586 lock operations and 6,361 condition synchronizations due to its pipelined image-processing stages, while **freqmine** reaches 990,025 lock operations because of shared FP-tree updates; **streamcluster** instead shows 129,600 barrier synchronizations caused by iterative clustering phases and global coordination points. Nevertheless, **n2d-highcpu** provides 4 GB of memory, one per vCPU, while **e2-standard-8** provides 32 GB, 4 GB per vCPU, and although the former has higher memory throughput, by collocating on it memory-intensive workloads such as **freqmine**, and **vips**, we have diminished the speedup gain slightly. In particular, the PARSEC native datasets report approximately 1 GB working-set requirements for **freqmine**, and hundreds of MB-scale image data for **vips**, all largely exceeding private cache capacity and forcing sustained DRAM traffic. However, on the other hand, **e2-standard-8** allocated resources are dynamically managed; as such, the underlying DRAM space at runtime is shared with other tenants. Conversely, LLC is bigger in **n2d-highcpu** compared to **e2-standard-8**, and the potential interference from tenants still applies. Accounting for these characteristic differences, our design generally trades off the risk of a noisy neighbour (mainly on LLC and memory) for a scheduling that prioritizes prevention of severe interferences among colocated jobs, and captures the steepest optimization speedup segment with the highest makespan reduction, obtained with parallel execution. Our second-best schedule was actually a completely sequential schedule, where no jobs run concurrently. It had a mean total time of 268.67s with a standard deviation of 2.62s. This is 20 seconds slower than our best schedule.

2. [34 points] Describe and analyze the AI-generated policy. [TODO: Xiaozhe]

(a) [17 points] Report the performance:

- Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes	42.33	48.00
blackscholes	27.67	30.33
canneal	93.00	87.33
freqmine	84.67	131.00
radix	38.00	37.00
streamcluster	144.33	151.67
vips	51.00	55.33
total time	258.00	344.67

- Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch

job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.

Answer:

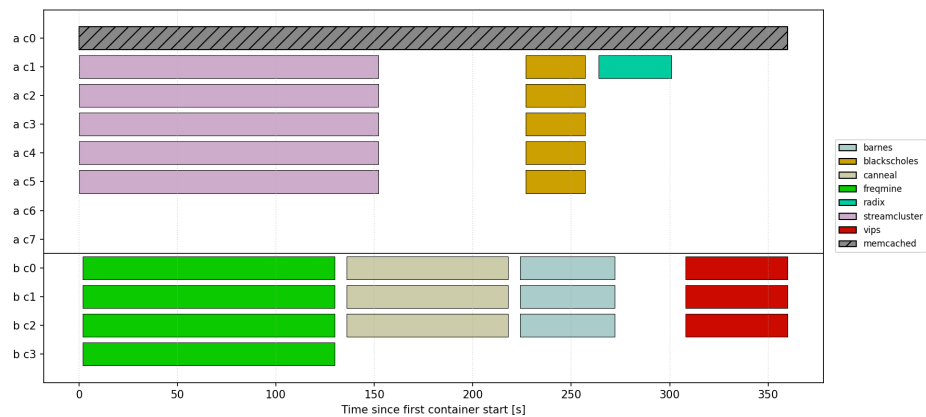


Figure 8: Run 1 schedule

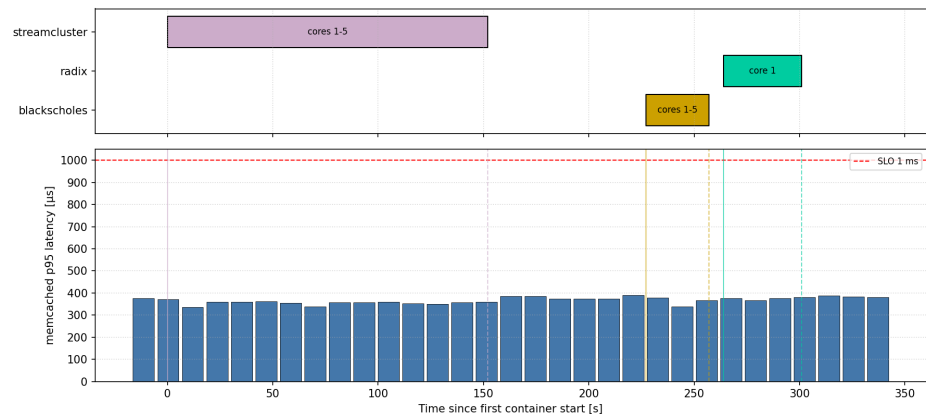


Figure 9: Run 1 memcache p95 latency

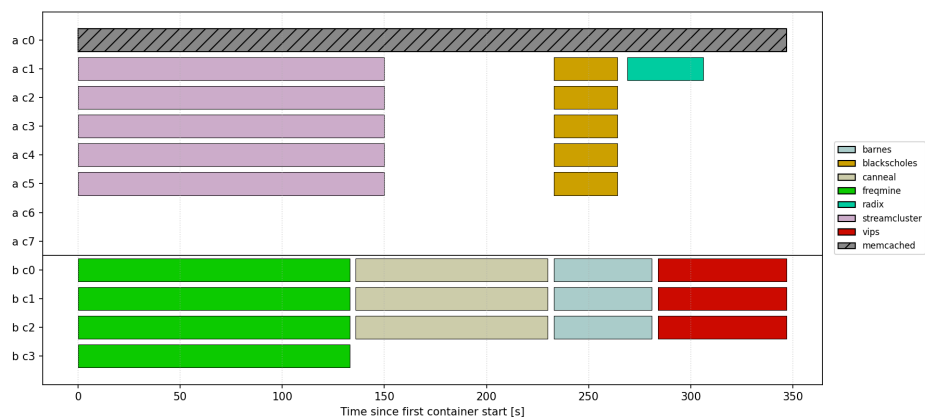


Figure 10: Run 2 schedule

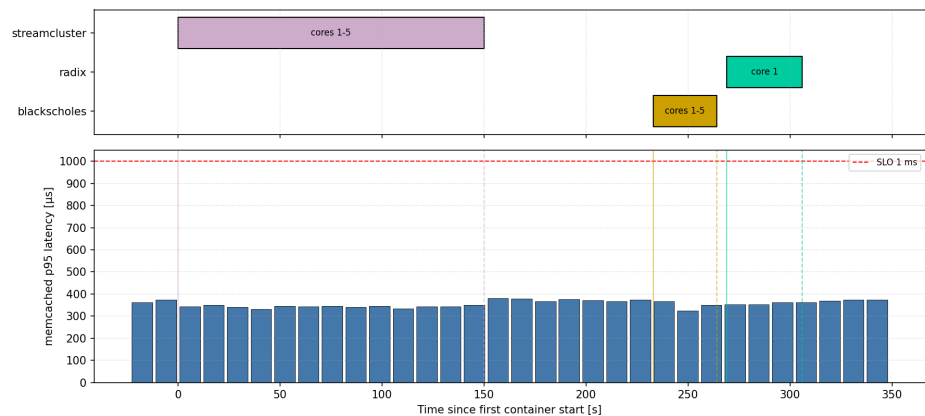


Figure 11: Run 2 memcache p95 latency

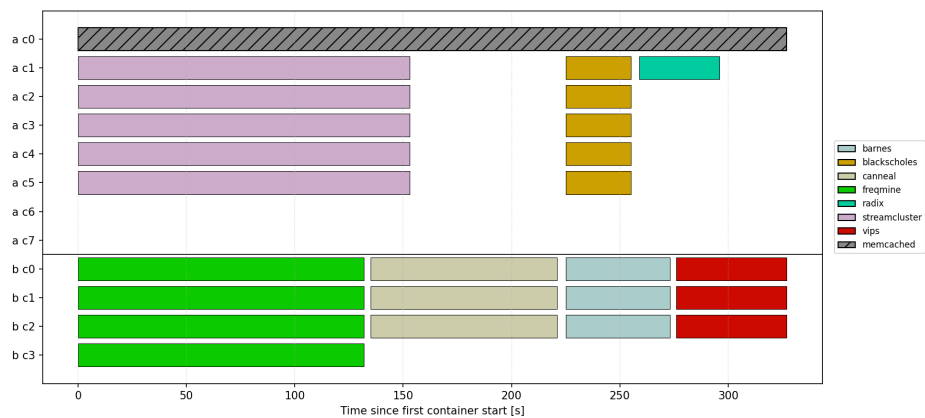


Figure 12: Run 3 schedule

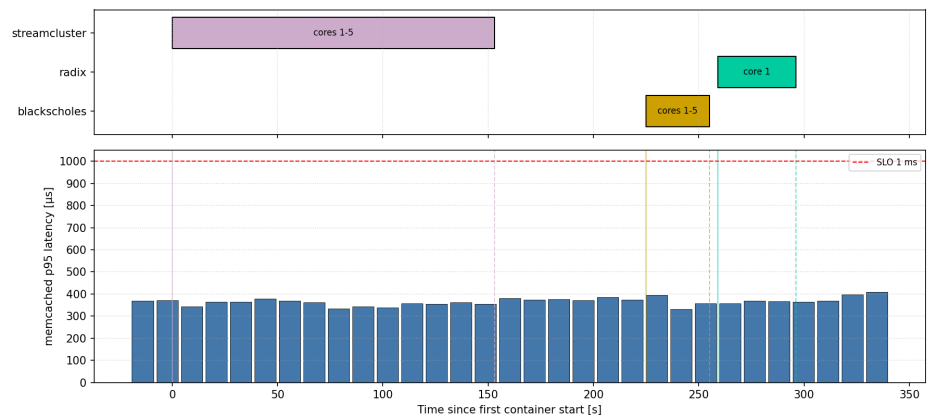


Figure 13: Run 3 memcache p95 latency

- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.

Answer:

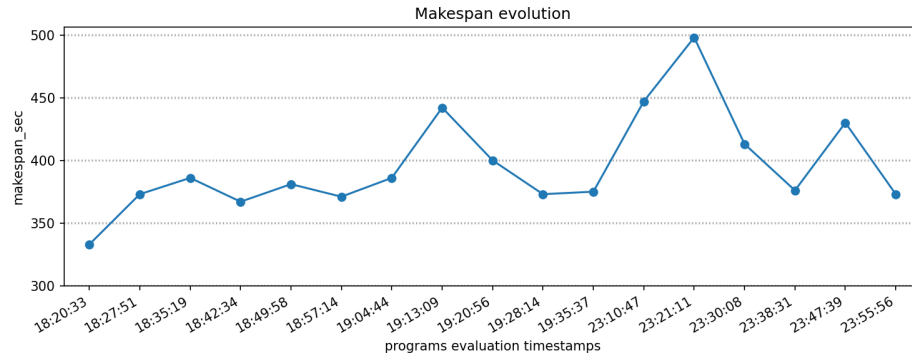


Figure 14: Makespan evolution

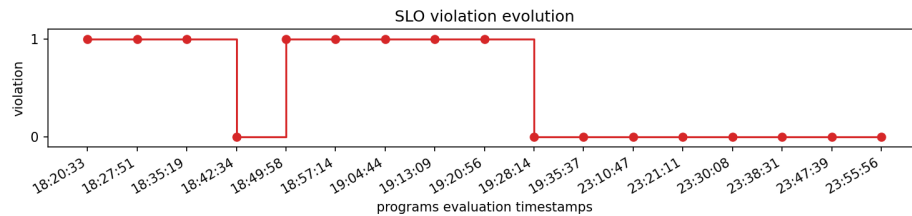


Figure 15: SLO violation evolution

- (b) [17 points] Analysis of the Evolutionary Process: Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?
- How do you measure success? For example, how do you design the ‘combined_score’ or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one “hallucination” or logical flaw the model produced during the process and how you identified it.

Answer: Following the order of questions:

- Evolution process: we ran 16 iterations in the final experiment batch that led to the presented best policy, though the total across all exploratory runs exceeds 100 iterations. This batch was interleaved by two configuration tweak breaks, during which we experimented with two models from the available Swiss AI platform: Apertus-70B and Qwen/Qwen3-Coder-30B-A3B-Instruct, as well as adjustments to temperature, and system message refinements. The latter included injections of workload-specific knowledge relevant for the evolution-phase, though these proved transient in effect. The model that produced the most structurally coherent mutations was Qwen3-Coder-30B-A3B-Instruct, used primarily for the final official batch.
- Optimization measurement: the scoring follows two principles. First, SLO compliance is non-negotiable, so any breach must collapse the score below the worst compliant run. Second, every makespan gain over the baseline should yield a meaningful score signal, which translated concretely to a steep increase rather than a flat one of the sigmoid function, used to model the combined score. Hence, the violation term carries heavy weight, with a hard cap that crushes any run breaching 1 ms regardless of speedup achieved. The sigmoid steepness k governs how sharply log-speedup maps into reward, and k is set to 4 specifically to increase the steepness of the curve, so small wins over the baseline produce visible score deltas instead of being squashed near 0.5. A couple of nuances worth flagging in our scoring system design:
 - The baseline makespan is bootstrapped from the first iteration’s measurement, serving as a placeholder anchor that the score curve is centered around.
 - The k value was tuned progressively, we observed a more gradual and leaner increase of a policy quality, as we raised k , evidencing a better interpreted significance of the metric to the LLM. However, increasing it toward 8 sharpens discrimination in a specific research domain and implies a premature saturation, we found a sweet spot at 4.
 - In the system message, we instruct the LLM to adopt either of two orthogonal approaches to the search according to the value of the parent’s combined score: ≥ 0.68 , the research should be breadth-first: modify significantly the policy on one of its core axis: node allocation, cores allocation, software threads, cores affinity, and concurrency structure ≥ 0.68 , the research must switch to depth-first: modifications are light tweaks over the same axis We have identified 0.68 as reliable threshold after consistent makespan reductions over several experiments.

Makespan speedup parameter

```
def _perf_speedup_term(*, measured_makespan_sec: float,
    baseline_makespan_sec: float, k: float = 4.0) -> float:
    if measured_makespan_sec <= 0 or baseline_makespan_sec <= 0:
        return 0.0
    log_speedup = math.log(baseline_makespan_sec / measured_makespan_sec)
    return 1.0 / (1.0 + math.exp(-k * log_speedup))
```

SLO violation parameter

```
def _slo_term(*, p95_series_ms: list[float], slo_ms: float = 1.0) -> float:
    if not p95_series_ms:
        return 0.0
    worst = max(p95_series_ms)
    violations = sum(1 for x in p95_series_ms if x > slo_ms)
    violation_fraction = violations / float(len(p95_series_ms))
```

```

if worst <= slo_ms:
    return 1.0
# Violator regime: always < 1.0; cap at 0.05.
persistence_term = 1.0 - violation_fraction
return 0.05 * persistence_term

```

- Metrics and observability system: The evaluator exposes three metric families: performance (makespan_sec, perf_term, baseline_makespan_sec, best_measured_makespan_sec, target_makespan_sec), SLO compliance (p95_worst_ms, slo_violation_fraction, slo_term), and structural diversity (node_allocation_scalar, dep_edge_count, parallelism_delta_node_a/b). Artifacts carry raw mcperf traces, pod timelines, effective job configurations, the spliced script path. Few common and critical faults are handled with EvaluationError exceptions: syntax errors, scheduler subprocess failures and timeout(600s) Giving the LLM complete exhaustive, and deep observability from scheduling decision to measured outcome.
- Initial baseline policy: The design had to navigate two opposing risks at once. Seeding too close to our best manual solution would trap the search in a local optimum. But seeding too naively would throw away everything learned in Parts 1 and 2, wasting runs on configurations that violate known constraints. Jobs' node placement follows the hard constraints emerged from experiments, depleting death paths, through we inflicted some structural changes that unbias the LLM:
 - `canneal` and `fregmine` are swapped across nodes compared to the best manual policy.
 - Each job is allocated to the full cores of its assigned node and the maximum thread count to match, deliberately ignoring what the profiling data and sets a clear and neutral default setting, which can be questioned in various directions.
 - Finally, ordering is kept fully sequential, such that almost any grounded jobs later scheduled as concurrent, will improve the makespan, and the score feedback stays meaningful right from the first iteration

Remarkably in the system message we specified these properties of the baseline scheduling.

The LLM is aware of the first base iteration since we are providing also a metric that express the first baseline run occurrence - `is_baseline_run`.

Moreover the baseline version of the evolvable snippet has the method that implements the in-place edit of job configurations(`edit_job_configurations()`), core of the LLM edits, encoding the baseline policy in a key-value and DAG data structures.

- Policy evolution: as aforementioned the baseline policy is deliberately naive, even though it seeds an high memcached latency due to collocation with `fregmine`, we wanted to anchor the exploration in a different starting point than the manual one. At the third iteration the LLM made a significant change, eventually the more fruitful structurally, by moving `fregmine` to `n2d-highcpu-4`, trimming blacksholes and streamcluster to 5 thread/cores(form 7), and rewiring the DAG with `fregmine` becoming third root, however the DAG remained structurally incoherent since `e2-standard-8` jobs were still dependant to `fregmine`. This cross-infra dependency persisted throughout almost half of following optimizations, and, anyways, set the base for the best policy. Which eventually resulted from minimal core/threads/affinity tweaks. The remaining half of optimizations attempted to update the structure, although keeping the same core shape, by for example allocating

blackscholes and radix on **n2d-highcpu-4** or running concurrently vips and barnes, evidencing some of the research directions explicated in the system message. Though, these changes revealed to be mostly detrimental for the performance, mainly due to an incomplete redesign that did not account for consequent dependencies upgrades. This optimization mode highlights the dominant failure pattern throughout the evolution: reallocating jobs across nodes without sanitizing the dependency graph. This failure mode emerged multiple times inconsistently in previous experiments as well, we have tried to decrease its risk by inserting a tailored verbatim that specifies how the primitives should be manipulated in the system message, however we were unsuccessful given the time constraints.

- Best policy analysis: the best evolved policy contains several internal incoherences that the manual one does not. Firstly, the dominant failure pattern covered above: blackscholes (**e2-standard-8**) waits on **canneal** (**n2d-highcpu-4**), and radix (**e2-standard-8**) waits on freqmine (**n2d-highcpu-4**) and streamcluster — meaning **e2-standard-8** sits idle while **n2d-highcpu-4** jobs run. The core pinning on **e2-standard-8** are themselves redundant with these deps: streamcluster (1–5), blackscholes (1–5) and radix (1) all overlap on the same cores, so they could never have run concurrently anyway, and cores 6–7 stay completely unused. On **n2d-highcpu-4** the opposite mistake appears: freqmine (0–3) and **canneal** (0–2) are both roots and launch simultaneously while contending on cores 0, 1, 2. These choices also disregard core characteristics of the workload profiles specified in the system message: firstly, streamcluster is the longest **e2-standard-8** job and still scales $4.62\times$ at 8T, yet the best policy caps it at 5T and abandons two cores, directly inflating the makespan. Secondly, **canneal** saturates the memory controller beyond 4T and was moved to **n2d-highcpu-4** only to be oversubscribed by freqmine on overlapping cores, so its $2.32\times$ at 4T scaling is lost to memory queue contention. The manual policy allocates **canneal** on **e2-standard-8** precisely because it is LLC-inert and safe beside memcached, freeing **n2d-highcpu-4** to host only the LLC-bound jobs, each given the full 4-core partition sequentially at $n=4$ to gain maximum multi-threading efficiency.
- Hallucinations: we have observed multiple hallucinations, across all LLMs, throughout experiments, which have been removed by explicit relevant instructions on the domain that was interpreted inaccurately:
 - cross-node job dependencies: regardless of the starting baseline policy and the instructions in the system message, the LLM mistakenly rearranged the DAG that encodes dependencies among jobs inserting edges between jobs on different nodes,
 - order-based cores specification in taskset pinning: even though it was specified in the system message that cores are identified with 0-based indexing, the LLM hallucinated cores range within the 1-8 range. Even though, we explicated the indexing and few valid jobs’ configuration args fields in the system message and replicated it in the method comment.
 - no-diff changes: in diff-based evolution, even though the system message mandated “at least TWO dimensions must change substantially” and listed the dimensions, the model consistently did not create any change, leaving commented-out backbone mutation logic, or produced leaf-value edits (± 1 thread/core), hallucinating compliance with the structural-change constraint.

- invalid syntax in full program changes: conversely, without diff-based changes, the model consistently hallucinated structurally invalid Python code, which presented a plethora of imperfections.
- Additional details: What proved to be significantly difficult throughout the experiments was steering the LLM toward structurally meaningful scheduler mutations, across all of the few models tested (Qwen/Qwen3-Coder-30B-A3B-Instruct, DeepSeek-Coder-33B, Apertus-70B, Qwen3-Coder-30B), and we acknowledge that very likely the same fault would emerge also on other, less capable, open weight models. Under diff-based evolution, changes were very superficial, mostly minimally tweaking threads/cores or affinity, and never touching concurrency topology or node allocation. Full-program rewriting produced syntactically invalid python code despite significantly updated job configuration logic. Custom MAP-Elites feature dimensions (node_allocation_scalar, encoding `e2-standard-8`location Hamming distance from baseline, and dep_edge_count, encoding scheduling concurrency degree) were introduced to drive population diversity, but their effect remained negligible given that all generated programs were structurally identical to the baseline. After several experiments, the strategy shifted to full-rewrite evolution with an explicit four-dimensional solution space (concurrency structure, node allocation, core/thread assignment, core pinning), enforcing structural departure as a hard constraint rather than a soft preference. Some core concrete techniques used:
 - Bug surface reduction: with full-rewrite evolution, we identify as the evolvable program only the relevant snippet, enclosed within `OPENEVOLVE COMMENTS`, and separate it in a distinct file, used as target in the `openevolve-run` program launch command. Which optimised version is afterward spliced by the evaluator into the immutable automation scaffold, confining the LLM’s rewrite surface to 60 lines.
 - Structured search space. The system message encodes unexplored configurations as a discrete (allocation, topology) decision tuple list, replacing open-ended family descriptions with enumerable, mutually exclusive options the LLM can select and track across iterations via artifacts.
 - Explicit exploration–exploitation phasing. Two search phases are defined in the system message: mandatory structural departure below a score threshold (exploration), then fine-grained parameter tuning above it (exploitation). This complements OpenEvolve’s native MAP-Elites diversity mechanism, which operates at the population level, with prompt-level guidance operating at the individual generation level.
 - Baseline anchoring. The `is_baseline_run` metric flags the first iteration explicitly, triggering the most aggressive structural change directive and preventing the LLM from treating the unmodified baseline as a valid departure point.

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the

characteristics of the first two parts of the project.

Part 4 [75 points]

1. [19 points] Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 2 \
    --scan 5000:125000:10000
```

a) [8 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. achieved QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

- Memcached with $T=1$ thread, $C=1$ core
- Memcached with $T=1$ thread, $C=2$ cores
- Memcached with $T=1$ thread, $C=3$ cores
- Memcached with $T=2$ threads, $C=1$ core
- Memcached with $T=2$ threads, $C=2$ cores
- Memcached with $T=2$ threads, $C=3$ cores
- Memcached with $T=3$ thread, $C=1$ cores
- Memcached with $T=3$ thread, $C=2$ cores
- Memcached with $T=3$ threads, $C=3$ cores

Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

Plots:

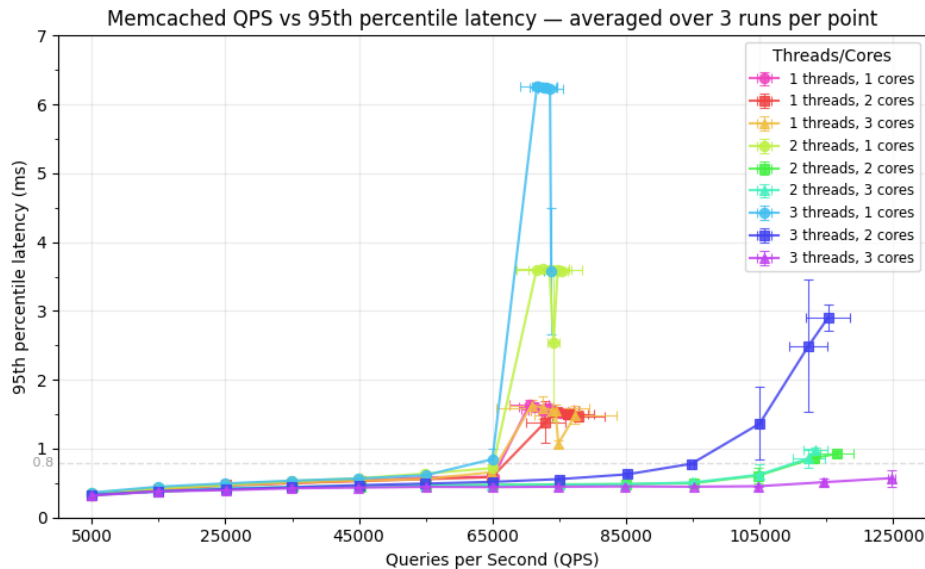


Figure 16: Memcached QPS vs p95 latency for different memcached configurations

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores and how this relates to the target QPS.

Summary: Memcached performance (i.e., highest actual QPS achieved) increases with the number of threads and cores, although to actually increase performance, the number of threads must be equal or higher than the number of cores. Additionally, we can notice that when the number of threads is higher than the number of cores, the latency increases and becomes more inconsistent.

b) [2 points] To support the highest load in the trace (around 125K QPS) without violating the 0.8ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

Answer: For this goal memcached requires 3 threads and 3 cores.

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you propose to use to guarantee the 0.8ms 95th percentile latency SLO while the load varies between 5K to 125K QPS?

Answer: We would use 3 threads as using any lower amount would make it impossible to achieve the goal, regardless of the number of cores allocated.

d) [8 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$, $C = 2$ and $C = 3$. Use the aforementioned `mcperf` command to sweep QPS from 5K to 125K.

Measure the CPU utilization on the memcached server at each load time step. Plot the performance of memcached using 1-core ($C = 1$), 2 cores ($C = 2$) and 3 ($C = 3$) in **three separate graphs**, for $C = 1$, $C = 2$ and $C = 3$, respectively. In each graph, plot achieved QPS on the x-axis, ranging from 0 to 125K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 0.8ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$, 200% for $C = 2$ and 300% for $C = 3$) on the right y-axis. For simplicity, we do not require error bars for these plots.

Plots:

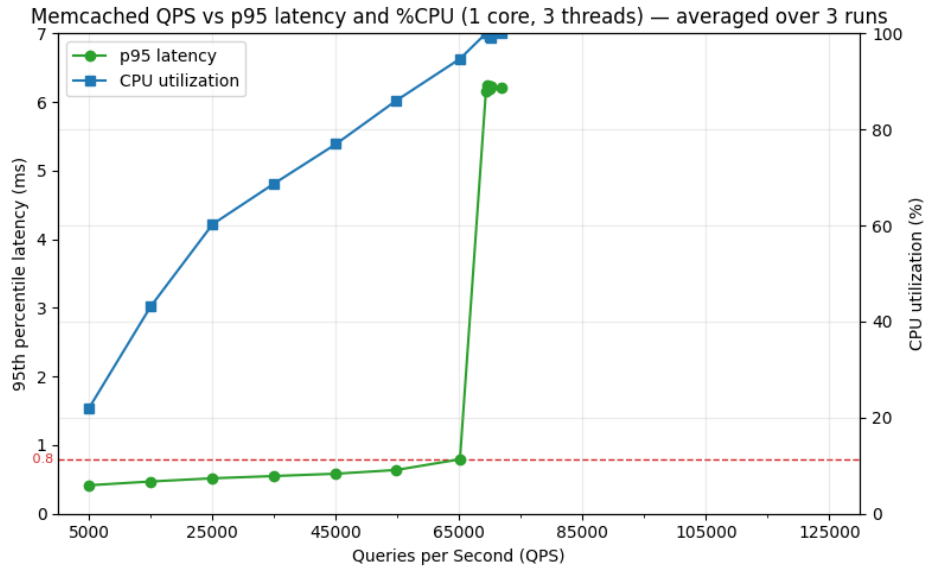


Figure 17: Memcached QPS vs p95 latency and %CPU for memcached with 1 core and 3 threads

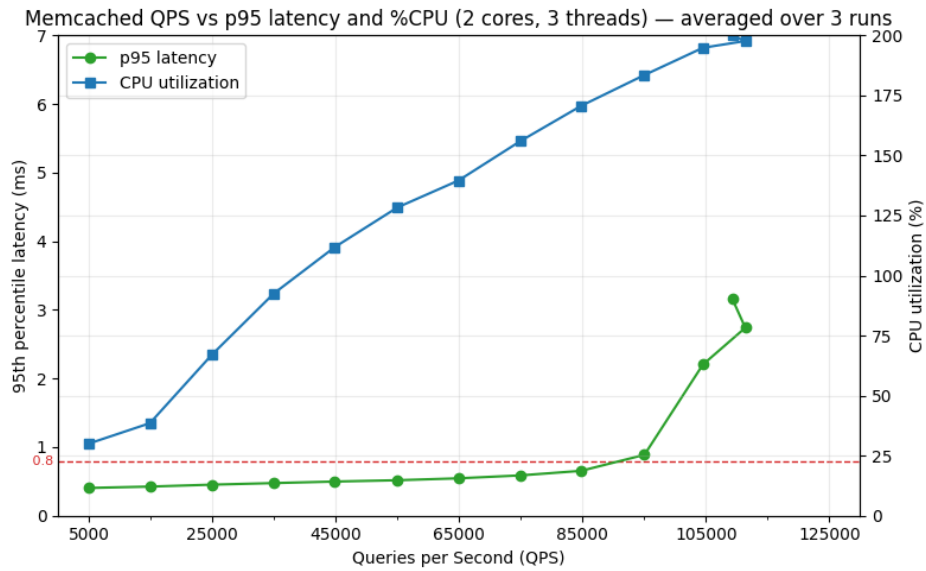


Figure 18: Memcached QPS vs p95 latency for memcached with 2 cores and 3 threads

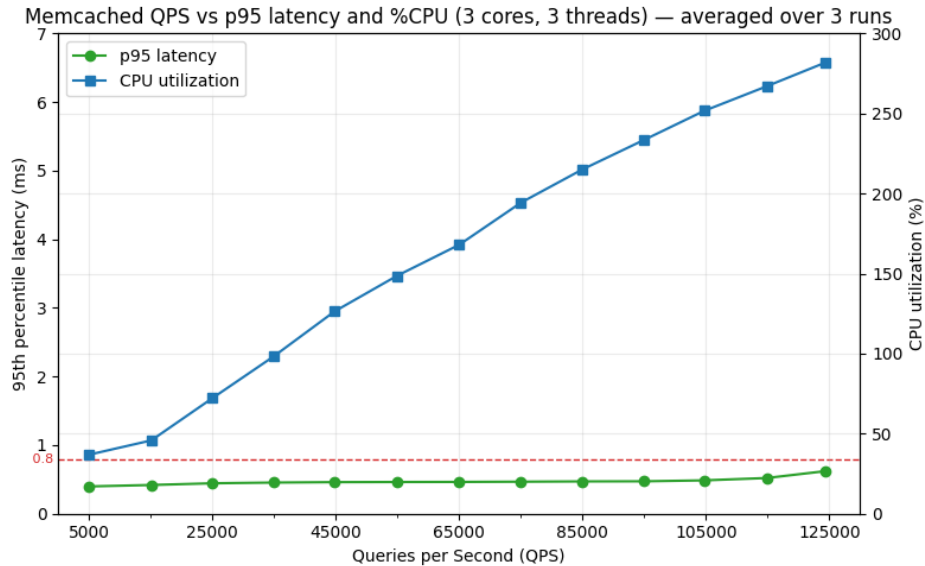


Figure 19: Memcached QPS vs p95 latency for memcached with 3 cores and 3 threads

2. [17 points] You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 110K in 15 second time intervals. Use the following command to run this trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
  --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
  --qps_interval 15 --qps_min 5000 --qps_max 110000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.

For this and the next questions, feel free to reduce the mcperf measurement duration (`-t` parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the benchmarks (batch jobs) on the 4-core VM. The goal of your scheduling policy is to successfully complete all batch jobs as soon as possible without violating the 0.8ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The batch jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the batch jobs complete successfully and do not crash. Note that batch jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):

Answer: Our scheduler runs only one task at each CPU core at a time. Core 0 is always used by memcached, and it can additionally get dynamically allocated cores 1 and 2, depending on CPU utilization. Then, on the free cores, the scheduler starts jobs one by one, one for each free core. Note that core 3 is always running jobs as it is never used by memcached. At any moment, the computer can be running between 1 and 3 jobs at the same time, depending on the number of cores used by memcached. When memcached requires a core, the container running the job on that core gets paused, and only resumes execution when memcached frees that core. The last job runs with 3 threads and can run on multiple cores, given that that core has finished running their job and is not being used by memcached.

- How do you decide how many cores to dynamically assign to memcached? Why?

Answer: Memcached gets between 1 and 3 cores depending on its total CPU utilization (ranging from 0% to 300%). We use threshold values for increasing or decreasing the amount of cores. These values were chosen based on the data obtained in part 4.1 d), such that they are close to the boundary of 0.8ms p95 latency, but leaving some safety margin. The threshold values for scaling up are the CPU utilization found when the load is around 20K QPS below the load that causes the p95 latency to cross the boundary. The values for scaling down were chosen similarly, when the load is around 30K QPS below, giving some safety room in between to avoid constantly scaling up and down under the same load. These values were chosen iteratively through experimentation, as we found our initial higher threshold values produced SLO violations.

core assignment	threshold
1 $\rightarrow$ 2	> 80
2 $\rightarrow$ 3	> 160
2 $\leftarrow$ 3	< 170
1 $\leftarrow$ 3	< 125
1 $\leftarrow$ 2	> 95

We found CPU utilization to be a very useful metric as it works as a proxy for the load that memcached is receiving, that also accounts for the innate variance. With the same load, memcached could sometimes perform without violating the SLO with 1 core, but other times it may violate it. Using CPU utilization compensates this as it reflects more accurately the stress it is under. We chose values that try to maximize CPU utilization but also ensure that we do not overload the core(s) and violate the SLO.

CPU utilization of memcached is measured in 0.5 second intervals. Whenever there is a change in the assignment of cores, the process waits 0.25 seconds before starting measuring again, allowing some time for the system to stabilize. `psutil` is used to measure the CPU utilization of the memcached process through `psutil.Process().cpu_percent(interval=None)`.

Even though there is a gap between the values for scaling up and down to avoid constantly increasing and decreasing cores under the same load, this still sometimes occurred. This worsened performance for both memcached and the job running as pausing and unpausing containers and reassigning memcached are not immediate processes. So another

mechanism was added that prevents rapid switches. Whenever the number memcached cores is scaled up, it can not be scaled down for the next 3 measurements (1.5 seconds), and vice versa. This system still allows for rapid scaling up from 1 to 3 cores and from 3 cores to 1 core, but avoids rapid switches back and forth caused by the system not stabilizing.

- How do you decide how many cores to assign each batch job? Why?

Answer: Each job gets 1 core except the last job, `streamcluster`, which gets at least 1 and at most 3 cores. This is because running one job on 2 cores (and 2 threads) does not bring a strict 2x speed-up, but slightly less. And as *n2d-highmem-4* has 64GB of memory, it is more than enough for running 3 jobs and memcached simultaneously. Thus, our scheduler only focuses on CPU cores, and the optimal assignment is to give one core to each job. `streamcluster` can scale up to 3 as it is the last job that will be running, and it can use the cores freed up by previous jobs.

- `barnes`: 1 core.
- `blackscholes`: 1 core.
- `canneal`: 1 core.
- `freqmine`: 1 core.
- `radix`: 1 core.
- `streamcluster`: Minimum 1 core, maximum 3 cores. Only one job can potentially run in multiple cores and `streamcluster` was chosen because of its long makespan and good parallel speedup, as shown by the experiment in part 2.2 a).
- `vips`: 1 core.

- How many threads do you use for each of the batch job? Why?

Answer: Each job, excepts for `streamcluster`, uses 1 thread as they will be running only at 1 core at a time.

- `barnes`: 1 thread.
- `blackscholes`: 1 thread.
- `canneal`: 1 thread.
- `freqmine`: 1 thread.
- `radix`: 1 thread.
- `streamcluster`: 3 threads as that gives the most performance when running at, potentially, 3 cores.
- `vips`: 1 thread.

- Which jobs run concurrently / are colocated and on which cores? Why?

Answer: Jobs run concurrently, on different cores, to other jobs. What runs with what can vary with the execution times of each job, and how many cores are taken up by memcached. Still, what runs with what is mostly determined by the order in which they run. As explained before, memory is not a problem, so the jobs may only compete for LLC. For this reason, the order of the jobs was chosen based on the results from part 2.1. At the beginning of the schedule, `barnes`, which is affected the most LLC interference, runs together with `freqmine` and `canneal`, jobs that are affected the least by LLC

interference.

- In which order did you run the batch jobs? Why?

Order: `freqmine`, `canneal`, `barnes`, `vips`, `blackscholes`, `radix`, `streamcluster`

Why: The reasoning behind the order of the first 3 jobs is given in the previous question. The next 3 jobs, `vips`, `blackscholes` and `radix` have similar makespans, and are affected similarly by LLC interference, so we believe the ordering between does not affect performance noticeably. Finally, `streamcluster` must run last to maximize its chances of using multiple cores

- How does your policy differ from the policy in Part 3? Why?

Answer: Our policy in part 4 is significantly different than our policy from part 3. In part 3 we minimize concurrent jobs while in part 4 we run up to 3 jobs concurrently. In part 3 we went for a multithreaded approach while in part 4 all but one job run on a single thread. The completely different approach is because they are completely different tasks with completely different resources. In part 3 we have 2 machines and a total of 12 cores and 36 GB of memory while in part 4 we have a single machine with 4 cores but 32 GB of memory. In part 3 memcached works just fine with 1 core out of 12, while in part 4 we have to scale it to take up to 3 out of 4 cores. As in part 4 the number of cores available is more limited and variable, we argue it is better to favour running jobs in single thread, to avoid the performance penalty that comes from running a job in less cores than the number of threads it uses.

- How did you implement your policy? e.g., docker cpu-set updates, taskset updates for memcached, pausing/unpausing containers, etc.

Answer: The scheduler is implemented in Python and uses the Python Docker SDK. It runs mainly on two threads. The main thread is in charge of starting new jobs as previous one finishes and cores are freed. When starting a job, it starts the container and uses `cpuset-cpus` to assign it to the free core. It then creates a daemon thread which uses `docker wait`. After the container finishes, the daemon thread is terminated after putting the container's name and exit code in an event queue that the main thread checks. The main thread then checks the exit code, logs, and tries to start the next job. If there are no more jobs to schedule, it assigns the free core to `streamcluster`.

The second thread does most of the heavy work by managing resources. It monitors memcached CPU utilization and uses the thresholds to decide when to scale up or down the number of cores that memcached uses. The reassignment of memcached cores is done using `taskset`. If memcached needs a core that is currently running a job, the container running on that core is paused using `docker pause`. Conversely, if memcached scales down and a core is free, it resumes the job running on that core using `docker unpause`. Towards the end of the execution, when `streamcluster` is already running, `docker update --cpuset-cpus` is used to give or revoke cores to the container running `streamcluster`. And while `streamcluster` is running on a single core, `docker unpause` and `docker pause` appropriately.

- Describe the design choices, ideas and trade-offs you took into account while creating

your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

Answer: The first scheduler we created took a different approach. It ran one job at a time but with 3 threads, and they could potentially scale up to 3 cores. The logic behind managing the resources was the same, although with more aggressive thresholds that caused multiple SLO violations. This scheduler performed reasonably okay, and was easier to implement. But we noticed it performed worse because it was sometimes running a job, with 3 threads, on a single core, taking a significant performance penalty compared to running the job with as many cores as threads. It was from this insight, and from the realization that memory is not a problem and that there was not noticeable interference between jobs, that we came up with our final scheduler.

3. [23 points] Run the following `mcperf` memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
    --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
    --qps_interval 15 --qps_min 5000 --qps_max 110000 \
    --qps_seed 2345
```

Measure memcached and batch job performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached. For each batch application, compute the mean and standard deviation of the execution time² across three runs. Compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes	166.84	1.11
blackscholes	107.02	1.90
canneal	250.57	8.21
freqmine	562.48	3.82
radix	42.49	2.12
streamcluster	327.78	4.90
vips	90.00	0.50
total time	804.76	8.05

Answer: We had 0 SLO violations throughout the 3 runs.

²Here, you should only consider the runtime, excluding time spans during which the container is paused.

Include three plots – one for each of the three runs – with the following information: The x-axis should be the time from execution start to end. Use three subplots that share the same x-axis. The first should clearly indicate for each core what job it is running at all time (e.g. using colored boxes). Use the colors proposed in this template (you can find them in `main.text`). If you pause/unpause jobs as part of your scheduling policy this should also be clearly visible. The second subplot should include two y-axis where the left one shows the current QPS and the right one should include the achieved p95 latency. Finally, the third subplot should plot the cpu utilization of each core separately. The readability of your plot will be part of awarded points.

Plots:

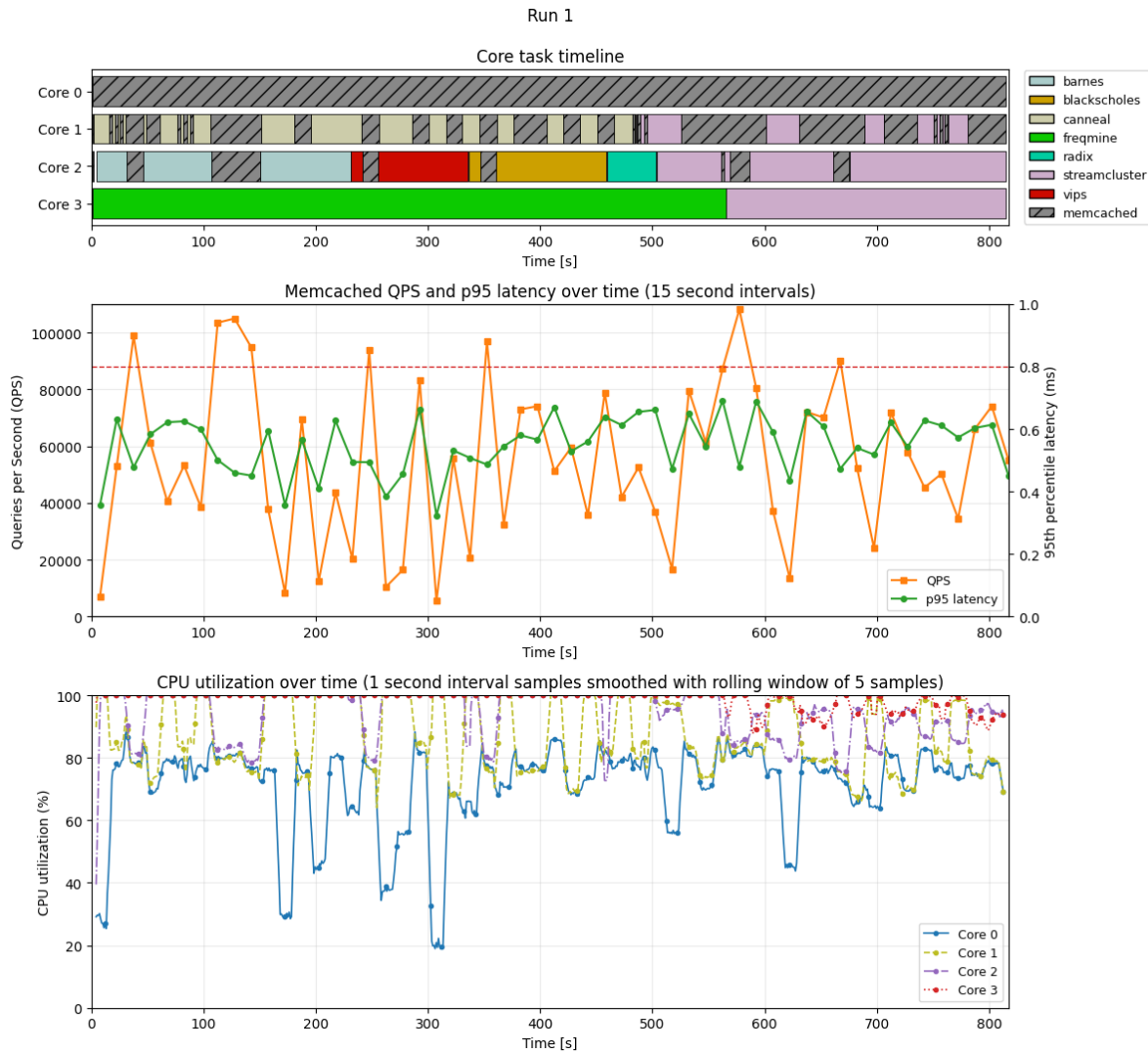


Figure 20: Run 1

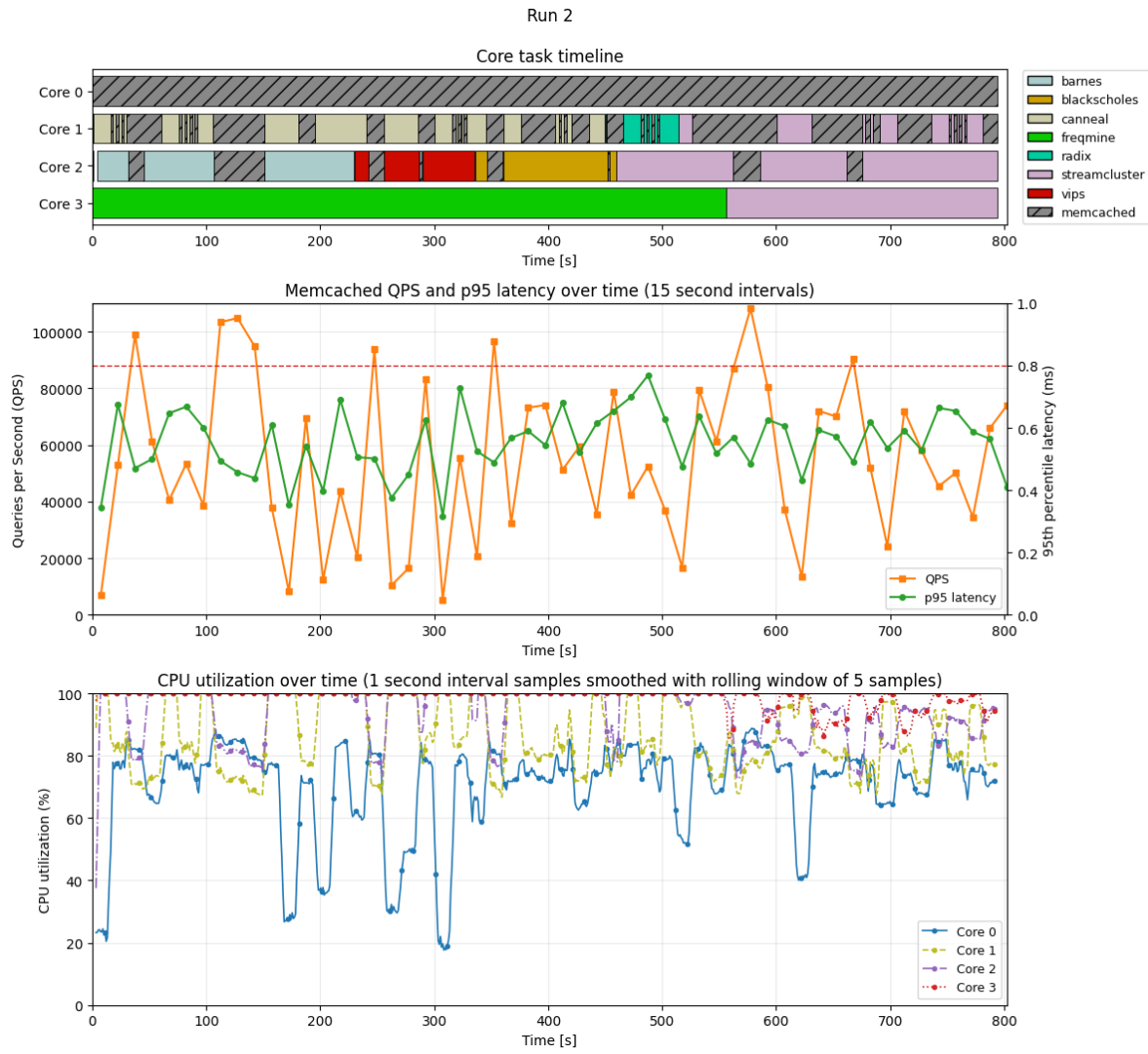


Figure 21: Run 2

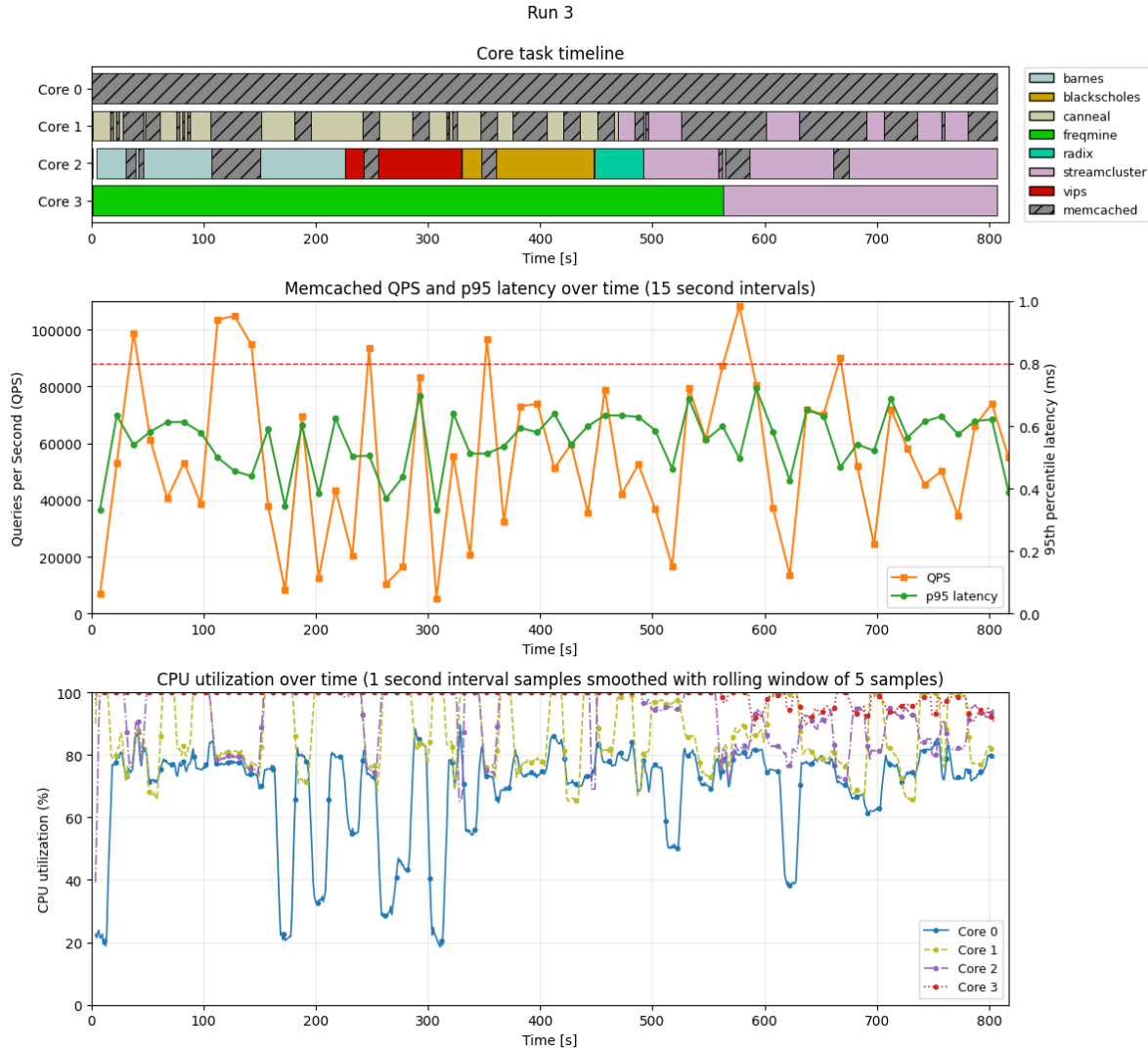


Figure 22: Run 3

4. [16 points] Repeat Part 4 Question 3 with a modified `mcperf` dynamic load trace with a 5 second time interval (`qps_interval`) instead of 15 second time interval. Use the following command:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
  --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
  --qps_interval 5 --qps_min 5000 --qps_max 110000 \
  --qps_seed 2345
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.

Summary: Total makespan with 5-second interval was 5% worse, we obtained an average of 846.13 seconds with a standard deviation of 6.27 seconds. This is around 40 seconds more

than with 15-second time intervals. The scheduler was able to finish within a reasonable time, but committed many SLO violations.

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency $> 0.8\text{ms}$, as a fraction of the total number of datapoints) with the 5-second time interval trace? The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

Answer: Over 3 runs, and 505 datapoints there were 87 SLO violations. That gives an SLO violation rate of 17.22%.

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?

Answer: We found experimentally that the smallest `qps_interval` that keep SLO violation rate under 3% is 11 seconds. All of our 3 were below that value, and in total we had 5 violations over 225 datapoints, giving a total SLO violation rate of 2.22%.

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.

Answer: We believe our scheduler reacts quickly to changes, taking measurements every 0.5 seconds, and usually taking between 1 and 2 measurements to adapt (sometimes 4 measurements when it is needed to scale from 1 to 3 cores). Despite that, p95 latency still spikes while the scheduler is adapting, and even though it is then kept under control fast, that initial spike skews the average. With just 8-second intervals we obtained an SLO violation rate of 4%, and the highest of those was just 1.041ms, not that far from the SLO. The reason why 11-second intervals are needed is just to allow the averages to "absorb" the initial spike. It does not help that also our scheduler thresholds are aggressive (for the sake of giving job execution more resources), and make the p95 latency be close to the SLO, even with 15-second intervals. With more conservative thresholds, that gave more resources to memcached earlier, we believe we could reduce the `qps_interval` needed to 8 seconds or even lower, at the cost of total job makespan.

Use of AI Tools: Did you use any AI tools for this project? If so, disclose them here and explain what you used the tool(s) for and why. Note that the use of AI tools is NOT needed and NOT expected for the project.

AI tools (i.e. Claude and ChatGPT) were used to assist with research, debugging and coding. All final ideas, arguments and code are the author's own. LLM outputs were reviewed, edited, and verified by the authors, who take full responsibility for the final content.